UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A PROJECT REPORT ENTITLED

DYNAMIC CONTEXT-AWARE TRIES FOR KNOWLEDGE GRAPH-CONSTRAINED LARGE LANGUAGE MODEL GENERATION

BY

BERNARD KIRK ADJANOR KATAMANSO

ERICA AMONOR

JOSEPH OSEI NYARKO

JESSICA AFUA ETORNAM NSAFOAH

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

PROJECT SUPERVISOR

.....

DR ERIC AFFUM

TARKWA, GHANA

August 2026

DECLARATION

We, BERNARD KIRK ADJANOR KATAMANSO, ERICA AMONOR, JOSEPH OSEI NYARKO and JESSICA AFUA ETORNAM NSAFOAH, declare that this project work is our own original work. It is being submitted for the degree of Bachelor of Science in Computer Science and Engineering at the University of Mines and Technology (UMaT), Tarkwa. It has not been submitted, either in whole or in part, for any degree or examination at any other university.

Name: KATAMANSO BERNARD KIRK ADJANOR

Signature: _____

Name: AMONOR ERICA

Signature: _____

Name: NYARKO JOSEPH OSEI

Signature: _____

Name: NSAFOAH JESSICA AFUA ETORNAM

Signature: _____

_____ day of _____ (year) _____

ABSTRACT

Knowledge Graph constrained decoding eliminates hallucination in large language model (LLM) reasoning by restricting output to verified paths. The leading approach, Graph Constrained Reasoning (GCR), achieves zero hallucination on Knowledge Graph Question Answering (KGQA) benchmarks by encoding all structurally valid paths in a static trie. The constraint is absolute but indifferent to the question: a three-hop query with twenty out-degree relations admits roughly 24,000 paths, and the LLM must navigate among them using the same internal knowledge that causes hallucination. We introduce Dynamic Context-Aware Tries (DCA-Trie), a dynamic constrained decoding framework that prunes the valid path set during generation using a TypeOracle, a lightweight symbolic module that checks two ontology gates per candidate triple. On a controlled 300-question WebQSP subset, DCA-Trie removes 13.8% of semantically irrelevant paths while maintaining a false negative rate below 5%. We further introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy, enabling independent evaluation of what the oracle filters versus what the LLM actually uses. SIR reveals a non-monotone relationship between constraint tightness and answer correctness: tighter constraints reduce the search space but do not improve Hits@1 (4.0pp lower for the static variant, 4.7pp for the lazy variant), while a per-hop architectural variant collapses by 25 points regardless of its gates. We show that the accuracy cost is attributable to the gates themselves via calibration, and that the lazy variant recovers the efficiency of dynamic adaptation in a single, stable decoding pass. DCA-Trie and SIR together provide the first framework for independently evaluating constraint quality and answer accuracy in knowledge graph-constrained decoding.

DEDICATION

Dedicated to our parents. For every sacrifice we didn't see and every kindness we never had to ask for. Thank you for everything.

ACKNOWLEDGEMENTS

We thank Dr. Eric Affum for his guidance, patience, and support throughout this project. His supervision was instrumental in shaping the direction of this work. We also express our appreciation to our families, friends, and mentors for their encouragement and understanding throughout the process. Finally, we acknowledge the academic community whose work has informed and inspired this research. This dissertation builds on the contributions of many scholars, and we are grateful for their work.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
BFS	Breadth-First Search
CoT	Chain-of-Thought
CWQ	ComplexWebQuestions
DCA-Trie	Dynamic Context-Aware Trie
DoG	Decoding on Graphs
FNR	False Negative Rate
FSM	Finite State Machine
GCD	Grammar-Constrained Decoding
GCR	Graph-Constrained Reasoning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
KG	Knowledge Graph
KGQA	Knowledge Graph Question Answering
LLM	Large Language Model
NLP	Natural Language Processing
QA	Question Answering
RAG	Retrieval-Augmented Generation
RLHF	Reinforcement Learning from Human Feedback
SBERT	Sentence Bidirectional Encoder Representations from Transformers
SIR	Semantic Irrelevance Ratio
SQL	Structured Query Language
VRAM	Video Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 Background

Despite their fluency, large language models (LLMs) are succeptible to hallucination, generating coherent and plausible text that is factually wrong (Ji *et al.*, 2023; Huang *et al.*, 2025). Autoregressive language models generate text by sampling tokens from a learned distribution conditioned on prior context, with no mechanism for grounding outputs against external knowledge sources during decoding (Lewis *et al.*, 2020). Errors introduced at one decoding step can propagate to subsequent steps, and in knowledge-intensive tasks such errors manifest not as stylistic infelicities but as factual inaccuracies. This makes factuality a central limitation of LLMs in precisely the settings where correctness is most critical.

A knowledge graph (KG) stores facts as triplets $(\text{entity}, \text{relation}, \text{entity})$ and supports multi-hop reasoning over verifiable connections (Bollacker *et al.*, 2008). Benchmarks such as WebQSP (Yih *et al.*, 2016) and ComplexWebQuestions (Talmor and Berant, 2018) test whether a system can trace these connections to answer natural language questions. Unconstrained LLMs, relying on parametric memory, frequently produce plausible but hallucinated reasoning trajectories over such graphs (He *et al.*, 2021; Lan *et al.*, 2022).

Retrieval-Augmented Generation (RAG) mitigates hallucination by injecting retrieved evidence into the context window (Lewis *et al.*, 2020; Izacard and Grave, 2021). However, retrieved evidence functions as a soft prompt rather than a hard constraint on generation: models can disregard relevant retrieved context or generate claims unsupported by the retrieved material when internal parametric knowledge overrides external evidence (Asai *et al.*, 2024).

1.2 Problem Statement

Constrained decoding takes a harder line by applying a validity mask over the output vocabulary. At each step, a constraint oracle forces the LLM to generate only tokens that correspond to valid paths in the KG (Willard and Louf, 2023; Geng *et al.*, 2023). Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025) achieves this with a pre-computed KG-Trie used as the oracle during beam search. The guarantee is structural in that, every generated token maps to a real graph edge. The problem is that the trie is built before decoding begins and never changes. It cannot adapt to the semantic intent of the question or the evolving reasoning chain, hence the LLM must still use its own judgment to filter out structurally valid but irrelevant paths (Li *et al.*, 2025; Luo *et al.*, 2025).

Decoding on Graphs (DoG) (Li *et al.*, 2025) addresses this rigidity by expanding the trie dynamically during decoding. But DoG admits all topologically adjacent nodes without semantic filtering. The result is unnecessary computational overhead and continued vulnerability to off-target paths.

These gaps are the gaps that DCA-Trie is designed to close.By conditioning the valid token set on three inputs, the knowledge graph, the input question, and the partially generated reasoning chain, DCA-Trie combines structural constraints with dynamic semantic filtering, aiming to reduce constraint permissiveness without sacrificing the faithfulness guarantees that make constrained decoding valuable in the first place.

1.3 Aim and Objectives

The aim of this project is to design, implement, and evaluate a Dynamic Context-Aware Trie (DCA-Trie) framework for knowledge graph-constrained LLM generation. By conditioning each decoding step on the knowledge graph, the input question, and the partial reasoning chain to improve multi-hop KGQA accuracy while maintaining 100% structural faithfulness to the underlying knowledge graph.

The specific objectives of this project work are to:

i. Characterize the constraint permissiveness problem in existing constrained-decoding approaches to KG-grounded reasoning;
ii. Design a semantic relevance scoring mechanism and a constraint oracle for filtering structurally valid but semantically irrelevant paths;
iii. Implement a static filtering variant that applies the oracle at trie construction time, and evaluate its impact on false negative rate;
iv. Implement a dynamic variant in which the trie is updated during decoding, conditioned on the question and the partially generated reasoning chain; and
v. Evaluate both variants against relevant baselines on standard knowledge-graph question answering benchmarks, and assess whether observed patterns generalize across benchmarks.

1.4 Tools and Facilities Used

1.4.1 Facilities

The following facilities were utilized in the execution of this project:

i. NVIDIA GeForce RTX 4090 GPU (24 GB VRAM) for all model inference and constrained decoding experiments.
ii. University library and online academic databases (ACM Digital Library, arXiv, Google Scholar, ACL Anthology) for literature review and citation verification.
iii. Shared team repository hosted on GitHub for version control, experiment logging, and reproducibility of all reported results.

1.4.2 Software Tools

Table 1.1 presents the software tools used in this project.

Table 1.1 Implemented Software Tools

Tool	**Use in Project**	**Rationale**
Python / HuggingFace Transformers	Model inference pipeline for Llama-3.1-8B, including the constrained decoding callback	Standard tokenization and generation utilities for the released GCR checkpoint
GCR Framework	Primary baseline; KG-Trie construction and graph-constrained decoding	Provides the trie-based decoding mechanism DCA-Trie extends
PyTorch	Deep learning framework underpinning inference and decoding logic	Supports SDPA attention and bfloat16 inference
marisa_trie	KG-Trie construction and prefix-based token lookup	$O(1)$ prefix lookup for the valid-token mask at each decoding step
NetworkX	Graph construction and path enumeration from question entities	Builds directed graphs from Freebase triplets
Freebase / WebQSP / CWQ	Knowledge graph source and KGQA benchmarks	Standard benchmarks for comparison with prior work
NVIDIA RTX 4090 (24GB)	GPU for all inference experiments	Sufficient VRAM for bfloat16 inference of an 8B-parameter model
VS Code, Git / GitHub	Development environment and version control	Coding, version tracking, and reproducibility

1.5 Scope and Delimitation

This project targets the permissiveness problem in GCR and DoG: existing constraint oracles admit every structurally reachable path regardless of relevance. Evaluation is restricted to multi-hop KGQA over Freebase benchmarks (WebQSP and CWQ). All experiments use GCR's publicly released Llama-3.1-8B checkpoint (Luo *et al.*, 2025); no fine-tuning is performed. Generalization to other KGs or non-KGQA tasks is identified as future work. All code and evaluation datasets will be made publicly available.

1.6 Organization of the Project Report

The rest of the report is organised as follows. Chapter 2 reviews literature on hallucination, knowledge graph question answering, and constrained decoding. Chapter 3 presents the research methodology, formalising DCA-Trie and its two variants. Chapter 4 reports results on WebQSP and CWQ, including ablation and error analysis. Chapter 5 presents conclusions and recommendations for future work.

CHAPTER 2

LITERATURE REVIEW

While large language models (LLMs) are good at many tasks, their use of autoregressive generation makes them likely to produce fluent but incorrect information (Ji *et al.*, 2023). For knowledge-intensive question answering, this unreliability is a major roadblock as the ability to verify reasoning against external facts is essential. To tackle this issue, recent studies have focused on grounding models in structured external facts through knowledge graphs. Still, keeping a model strictly aligned with these graphs during generation, without losing its natural flow, is a difficult challenge.

This literature review looks at the current research on these challenges and examines how knowledge graph constraints might help ensure factual accuracy during multi-hop reasoning. This chapter outlines the connections between language model hallucination, multi-step validation, and structured grounding. By assessing current strategies to reduce errors, from improved prompting to fixed decoding, this chapter shows where soft-grounding methods do not meet expectations, highlighting the specific technical gaps this project addresses.

2.1 Large Language Models and Hallucination

Modern LLMs are built on the Transformer architecture (Vaswani *et al.*, 2017), using scaled dot-product self-attention to weight token importance regardless of positional distance. The field has largely moved toward decoder-only models with causal masking, where each token attends only to preceding tokens. The Llama-3.1-8B model (Dubey *et al.*, 2024) is a prominent example of this design and has been adopted as the backbone in recent KG-constrained decoding work (Luo *et al.*, 2025).

Text production in these models is fundamentally autoregressive: given a vocabulary $\mathcal{V}$, an input x, and prior tokens $y_{<t}$, the model computes $P(y_t|y_{<t}, x)$ over the full vocabulary. Since this distribution is always defined over every token, any token has a positive, non-zero chance of being generated at any step. Decoding strategies such as beam search and nucleus sampling (Holtzman *et al.*, 2020) determine how a token is chosen from this distribution.

This autoregressive nature creates fluent, coherent text, but since each step relies on prior outputs without an external verifier, it also directly causes hallucination.

LLMs are pretrained on vast unlabelled text using next-token prediction. The GPT series (Brown *et al.* 2020; OpenAI 2024) and the Llama series (Touvron, Martin, and Stone 2023; Dubey *et al.* 2024) show that scaling this simple task leads to powerful downstream capabilities. During pretraining, the model internalizes patterns and facts from its training data, but this memory is fundamentally static and statistical. The model lacks a way to interact with dynamic external facts or ensure that its outputs reflect verified truths.

2.1.1 Definition and Structural Causes

Hallucination in natural language generation refers to the production of content that is fluent and internally coherent yet factually unsupported by or in contradiction with verifiable sources (Ji *et al.*, 2023). Huang *et al.* (2025) refine this into *intrinsic hallucination* (output contradicts provided source context) and *extrinsic hallucination* (output contains claims unverifiable against any source). Both types appear at non-trivial rates: Ji *et al.* (2023) survey over 30 works finding hallucination rates from 15% to over 60%, and Luo *et al.* (2025) report that retrieval-augmented reasoning approaches produce hallucinated reasoning steps in approximately one-third of cases even with direct access to a knowledge graph.

Hallucination is generally attributed to a combination of data, training, and inference-time factors (Huang *et al.*, 2025). This work is concerned with the last of these: the autoregressive decoding mechanism itself makes hallucination structurally possible, independent of data quality or training procedure. Three properties of this mechanism are relevant. First, the generation distribution is based entirely on learned parameters, with no component that verifies tokens against an external source. Second, errors propagate forward: a hallucinated token at step t becomes conditioning context for step $t+1$, producing fluent continuations of an incorrect chain. Third, the model imposes no hard constraint on which tokens are eligible at each step, so factually invalid continuations remain within the support of the output distribution (Brown *et al.*, 2020).

2.1.2 The Scaling Paradox

A common argument is that larger models should eventually achieve factual reliability. Evidence does not support this. Lin, Hilton, and Evans (2022) introduce TruthfulQA and find that larger models do not outperform smaller ones in truthfulness and in some categories show *inverse scaling*, reproducing widespread incorrect beliefs more effectively. Perez *et al.* (2023) show that RLHF-trained models generate confident-sounding responses aligned with apparent user expectations rather than verifiable facts. Kandpal *et al.* (2023) demonstrate that larger models improve on frequently-seen entity combinations but not on compositional questions requiring unseen fact combinations, precisely the structure of multi-hop KG questions. Mallen *et al.* (2023) confirm significant hallucination rates for GPT-4 on tail-entity questions regardless of model size. Together, these findings establish hallucination as a structural feature of the autoregressive mechanism, not a scaling issue.

2.2 Chain-of-Thought and Its Limits

2.2.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting (Bosma *et al.*, 2022) is the dominant paradigm for eliciting multi-step reasoning from LLMs. By including worked examples that show intermediate reasoning steps, the model learns to decompose complex problems into simpler sub-problems. Bosma *et al.* (2022) demonstrate accuracy improvements on arithmetic, commonsense, and multi-hop QA benchmarks, with the capability emerging at approximately 100 billion parameters. Zero-shot chain-of-thought (Kojima *et al.*, 2022) achieves similar benefits by adding "Let's think step by step" to the prompt.

Despite these improvements, CoT has a fundamental structural limitation: each intermediate step is generated by the same autoregressive mechanism. An incorrect step propagates forward as conditioning context, and no mechanism verifies that any step corresponds to a fact in an external source. A CoT chain can be entirely fluent and internally coherent while containing factually fabricated steps.

2.2.2 Self-Consistency and Tree-of-Thought

Self-consistency (Wang *et al.*, 2023) samples several independent CoT chains and chooses the final answer by majority vote, reducing sensitivity to individual hallucinated chains. However, a majority of independently hallucinated chains can still produce an incorrect answer, especially on deep multi-hop questions. Tree-of-Thought (Cao *et al.*, 2023) treats generation as search through a tree of partial reasoning sequences, using the LLM as both generator and evaluator. The key limitation persists: pruning decisions rely on the model's own parameters without external verification of factual accuracy.

2.2.3 Why Prompting Cannot Guarantee Faithfulness

All prompting-based methods (few-shot, CoT, self-consistency, tree-of-thought) modify the *input* to the LLM without modifying the *generation mechanism*. Since decoding happens over the full vocabulary, each token has a strictly positive probability at every step. True structural faithfulness requires that invalid tokens receive exactly zero probability. This can only be achieved by directly changing the decoding algorithm (Geng *et al.*, 2023; Willard and Louf, 2023).

2.3 Knowledge Graphs and Knowledge Graph Question Answering

2.3.1 Knowledge Graphs

A knowledge graph is a structured database of verified facts, represented as directed, labelled triplets (e, r, e') where $e, e' \in \mathcal{E}$ are entities and $r \in \mathcal{R}$ is a relation type. Figure 2.1 illustrates a simplified fragment of such a structure. Freebase (Bollacker *et al.*, 2008), the KG underlying the evaluation benchmarks used in this work, encodes tens of millions of triplets across geography, history, entertainment, and science (Yih *et al.*, 2016; Talmor and Berant, 2018).

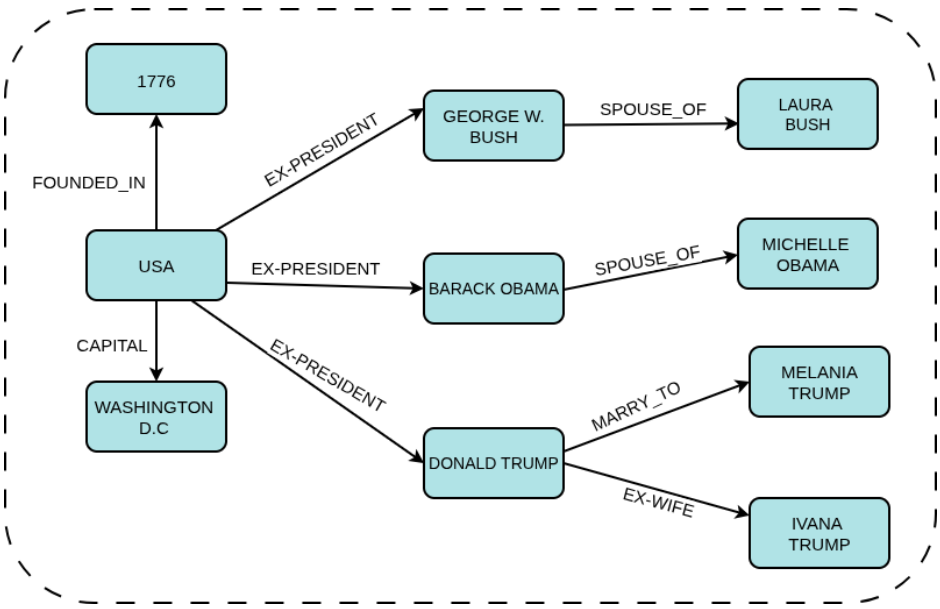

Figure 2.1 Knowledge Graph Fragment

The critical distinction between a KG and an unstructured text corpus is that every triplet has been explicitly verified and is machine-readable in deterministic form. This makes KGs suitable as sources for hard constraints on generation: a candidate token can be verified via exact lookup rather than probabilistic semantic matching. A multi-hop reasoning path of length L is a sequence $(e_0, r_1, e_1, \ldots, r_L, e_L)$ in which each consecutive triplet (e_{i-1}, r_i, e_i) is a verified edge of $\mathcal{G}$ (Lan *et al.*, 2022).

2.3.2 Knowledge Graph Question Answering

KGQA requires a system to identify the answer entity e_L to a natural language question q by reasoning through a chain of verified KG triplets. Formally: given question q with question entities $\mathcal{E}_q \subseteq \mathcal{E}$ identified by entity linking, identify the terminal entity of the correct reasoning path in $\mathcal{G}$ (Lan *et al.*, 2022). Early approaches relied on semantic parsing, information retrieval over KG subgraphs, or reinforcement-learning-based path traversal (Das *et al.*, 2018). More recent approaches use LLMs as retrievers, agents iteratively querying the KG and revising partial answers (Mavromatis and Karypis, 2022), generators conditioned on retrieved subgraph context (Jiang *et al.*, 2023a; Jin *et al.*, 2024; Luo *et al.*, 2024), unified retrieval-and-reasoning pipelines (Jiang *et al.*, 2022), or KG-infused pretrained representations (Wang *et al.*, 2021). The move toward LLM-based KGQA has improved fluency but brought back hallucination risks.

2.3.3 Evaluation Benchmarks

WebQSP (Yih *et al.*, 2016) contains 4,737 questions on Freebase with reasoning depths of one to two hops, evaluated using Hits@1 and F1. ComplexWebQuestions (CWQ; Talmor and Berant 2018) builds on WebQSP through conjunction, composition, and comparative operations, resulting in about 34,000 questions requiring up to four hops. The two benchmarks stress different aspects of path selection: WebQSP questions reach the answer in one or two hops, so the valid path set stays small; CWQ questions reach up to four hops, so the valid paths are longer and correct selection depends on sustaining the right trajectory across more steps, not on filtering a larger candidate set.

2.3.4 Multi-Hop Complexity

For an entity with average out-degree d in the KG, the number of valid paths of length L from the question entities increases as $|\mathcal{E}_q| \times d^L$. For well-connected Freebase entities where $d \approx 20$ and a question may have up to three question entities, a three-hop question can generate up to 24,000 structurally valid paths, but only one leads to the correct answer (He *et al.*, 2021; Lan *et al.*, 2022). At each step, the model must choose the correct next token from a vast array of valid yet incorrect options.

2.4 Constrained Decoding for Faithful Text Generation

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting, which changes the input to the LLM, constrained decoding alters the token selection process itself.

2.4.1 Format-Constrained Decoding

Logit masking (Geng *et al.*, 2023; Willard and Louf, 2023) computes a binary mask over the vocabulary at each generation step, setting the log-probability of invalid tokens to $-\infty$ before the softmax. Geng *et al.* (2023) constrain generation to strings conforming to a context-free grammar via an Earley parser oracle. Willard and Louf (2023) reduce the computational cost with an efficient finite-state machine (FSM) formulation, precomputing an index from automaton states to valid vocabulary subsets for amortised $O(1)$ lookup. Additional methods include Synchromesh (Poesia *et al.*, 2022) for program synthesis, PICARD (Scholak, Schucher,

and Bahdanau, 2021) for schema-constrained SQL parsing, NeuroLogic Decoding (Lu *et al.*, 2021) for propositional logic constraints, and GENRE (De Cao *et al.*, 2021) for trie-constrained entity retrieval. The shared limitation of all these approaches is that they enforce output *format* rather than *factual content*. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself.

2.4.2 Knowledge Graph-Constrained Decoding

Applying constrained decoding to knowledge graph faithfulness uses the KG itself as the constraint oracle. At each generation step, only tokens corresponding to valid continuations of a KG reasoning path are allowed, making hallucination of KG facts structurally impossible.

Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025) builds a KG-Trie (a prefix tree of all KG paths within L hops of the question entities, created by BFS before generation) and uses it as the constraint oracle during beam-search decoding. GCR achieves 92.6% Hits@1 on WebQSP and 75.8% on CWQ with 100% structural faithfulness, using a Llama-3.1-8B backbone. The trie is created once and remains unchanged: the valid token set is fixed regardless of what has been generated.

Decoding on Graphs (DoG) (Li *et al.*, 2025) presents step-wise trie expansion as a partial solution to GCR's static oracle. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity, making the oracle topologically dynamic. DoG achieves 90.99% Hits@1 on WebQSP and 76.08% on CWQ with 100% structural faithfulness. However, the expansion rule includes all KG neighbors regardless of relevance to the question, and step-wise trie construction compromises the efficiency of precomputed structures.

RouterKGQA (Yuan *et al.*, 2026) routes between a specialized KG-reasoning model and a general LLM based on question complexity. It uses SBERT embeddings (`nomic-embed-text-v1`) to filter candidate answers after generation. RouterKGQA's semantic scoring operates at the answer selection level rather than at the token constraint level: paths are created first and filtered afterward.

2.4.3 Beam Search and Trie-Based Constraints

Beam search (Lowerre, 1976; Holtzman *et al.*, 2020) keeps the k most promising partial hypotheses at each step, reducing early-error multiplication. A prefix trie (Fredkin, 1960; Willard and Louf, 2023) yields the valid next-token subset in $O(1)$ time. GCR's KG-Trie (Luo *et al.*, 2025) serializes verified multi-hop paths from the KG into string format mapped to the LLM's token vocabulary. When beam search is merged with a KG-Trie oracle, the decoding process becomes a strictly bounded traversal of the prefix tree: at each autoregressive step, the current beam hypotheses map to specific trie states, and a mathematical mask forces the probability of any token not present as a child node to zero. The beam search then advances strictly down permissible branches.

2.5 Retrieval and Soft Grounding

Retrieval-augmented generation (RAG) (Lewis *et al.*, 2020) is the leading approach for grounding LLM outputs in external knowledge. A dense retriever selects relevant passages based on embedding similarity, and a reader LLM generates the answer from the retrieved context. Fusion-in-Decoder (Izacard and Grave, 2021) enhances this by encoding multiple passages separately and merging them in the decoder. Self-RAG (Asai *et al.*, 2024) adds reflection tokens for when to retrieve and whether passages support claims. FLARE (Jiang *et al.*, 2023b) triggers retrieval when next-token confidence drops. IRCoT (Trivedi *et al.*, 2023) interleaves retrieval with CoT steps, improving multi-hop accuracy. GraphRAG (Edge *et al.*, 2024) builds community-structured KGs from document collections for complex relational queries.

In the KGQA context, several approaches enhance RAG with structured context. He *et al.* (2021) retrieve KG subgraphs with attention-guided traversal. Yasunaga *et al.* (2021) combine text passages and KG neighborhoods through graph neural networks. Shi *et al.* (2021) retrieve KG paths as natural language context strings. Sentence-BERT (Reimers and Gurevych, 2019) encoders produce dense vector representations where cosine similarity proxies semantic alignment, applied in these systems to rank candidate KG paths as soft retrieval signals (Shi *et al.*, 2021; Saxena, Chakrabarti, and Talukdar, 2022).

2.5.1 The Structural Limitation of Soft Grounding

Despite their empirical improvements, all RAG approaches share a structural limitation: retrieved context is a soft influence on generation. As shown in Figure 2.2, the retriever determines what the LLM sees; it does not determine what the LLM is permitted to generate. Because the generation mechanism remains unchanged, there exists a nonzero probability of generating any token at any step, including tokens that correspond to no verified KG fact. Structural faithfulness with probability one cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Luo *et al.*, 2025; Geng *et al.*, 2023).

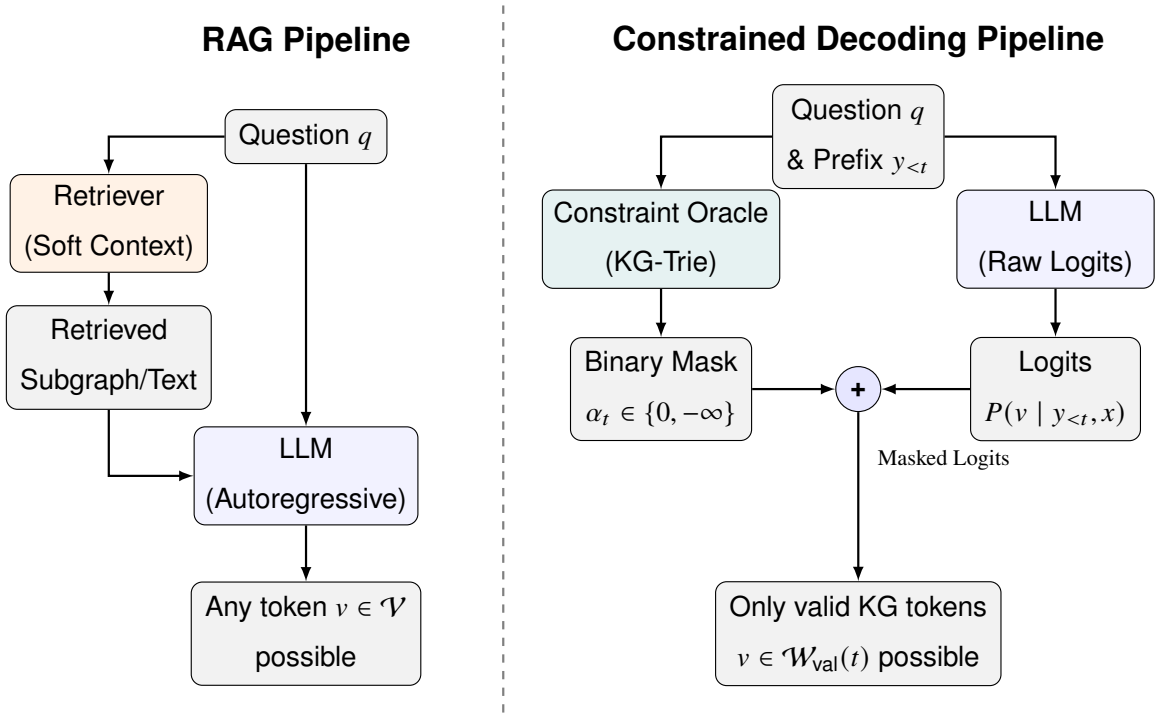

Figure 2.2 Standard RAG vs. KG-Constrained Decoding

2.6 Unresolved Gaps

The review shows that two different research paths, constrained decoding for structural faithfulness and semantic similarity for relevance-guided reasoning, have developed mainly in parallel without being combined at the constraint-oracle level. Constrained decoding methods (Luo *et al.*, 2025; Li *et al.*, 2025) achieve structural faithfulness but create their constraint oracles without considering question semantics or the context of generation. This leads to valid token sets that include many irrelevant paths. Semantic similarity methods (Shi

et al., 2021; Saxena, Chakrabarti, and Talukdar, 2022; Yuan *et al.*, 2026) improve relevance but act as soft signals that affect retrieval or selection after the fact, without changing the generation process.

Three specific gaps emerge from this review.

Gap 1: The Permissiveness Problem. Current KG-constrained decoding frameworks build their constraint oracles based on graph structure and question entities only. They do not consider question semantics or the current generation state. In the case of multi-hop questions, this results in valid token sets with thousands of structurally correct but semantically irrelevant paths at each generation step. The LLM has to sort through these paths using its own knowledge, which reintroduces the risk of hallucination that constrained decoding aimed to eliminate (Luo *et al.*, 2025; Li *et al.*, 2025).

Gap 2: The Static-Dynamic Efficiency Tradeoff. GCR's precomputed static trie is efficient in computing but lacks semantic awareness and remains the same at every generation step. DoG's step-wise topological expansion adds some dynamism but requires reconstructing the trie at every step and does not include semantic filtering (Li *et al.*, 2025; Willard and Louf, 2023). No current framework achieves semantic responsiveness, where the valid set narrows progressively as reasoning moves in a specific direction, while also maintaining the efficiency of precomputed constraint structures.

Gap 3: The Absence of a Constraint Quality Metric. No existing work offers a metric that measures constraint permissiveness separately from the accuracy of downstream answers. Without this metric, it is not possible to track progress on reducing irrelevant paths in the valid set or to compare frameworks based on constraint quality without factoring in model quality.

Table 2.1 summarizes the key works reviewed in this chapter, categorizing them by their approach, whether they provide structural faithfulness guarantees, and whether they incorporate semantic conditioning into their constraint mechanisms.

Table 2.1 Summary of Key Related Works

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Luo *et al.* (2025)	Static KG-Trie decoding	Yes (100%)	No
Li *et al.* (2025)	Step-wise topological expansion	Yes (100%)	No
Yuan *et al.* (2026)	Routing + post-hoc filtering	Probabilistic	Yes (answer-level)
Luo *et al.* (2024)	Reasoning path planning	No	Partial
Jiang *et al.* (2023a)	LLM over structured data	No	Partial
Jin *et al.* (2024)	Graph-augmented CoT	No	Partial
Jiang *et al.* (2022)	Unified retrieval and reasoning	No	Partial
Wang *et al.* (2021)	KG-pretrained language model	No	Partial
Mavromatis and Karypis (2022)	Iterative reasoning revision	No	Partial
He *et al.* (2021)	KG subgraph attention	No	Partial
Yasunaga *et al.* (2021)	Text-KG graph neural network	No	Partial
Das *et al.* (2018)	RL-based KG path traversal	No	Soft
Saxena, Chakrabarti, and Talukdar (2022)	Embedding-based path selection	No	Soft
Shi *et al.* (2021)	Path retrieval by similarity	No	Soft (retrieval)
Asai *et al.* (2024)	Retrieval with reflection	No	Soft
Edge *et al.* (2024)	Community-structured retrieval	No	Soft

Work	**Approach**	**Structural Faithfulness**	**Semantic Conditioning**
Geng *et al.* (2023)	Grammar-constrained decoding	Format only	No
Willard and Louf (2023)	FSM-indexed constrained decoding	Format only	No
De Cao *et al.* (2021)	Entity trie-constrained decoding	Entity names only	No
Scholak, Schucher, and Bahdanau (2021)	Schema-constrained SQL parsing	Schema only	No
Lu *et al.* (2021)	Logic-constrained decoding	Format only	No

These three gaps collectively identify an open problem at the intersection of the two research trajectories reviewed in this chapter: integrating semantic relevance scoring directly into the constraint oracle, conditioning the valid token set jointly on the knowledge graph, the question semantics, and the evolving generation state.

CHAPTER 3

METHODOLOGY

3.1 Overview

This chapter formalises the permissiveness problem identified in Chapter 2 and presents the Dynamic Context-Aware Trie (DCA-Trie) framework as a solution. The upgraded oracle specification and the Semantic Irrelevance Ratio (SIR) are defined first, with SIR serving as a diagnostic metric that measures constraint quality independently of answer accuracy. The chapter then traces the research evolution from an initial cosine-similarity scorer through a decomposed product score to the final symbolic TypeOracle, explaining the rationale behind each design choice. The static pre-filtering design of v1 and the step-wise dynamic expansion design of v2 are described using the TypeOracle mechanism. The chapter closes with the baseline configurations, evaluation protocol, and scope boundaries that govern the experimental work in Chapter 4.

3.2 Formal Problem Specification

3.2.1 Core Limitation of Existing Frameworks

As established in Chapter 2, current graph-constrained frameworks (notably GCR (Luo *et al.*, 2025) and DoG (Li *et al.*, 2025)) rely on a deterministic oracle that looks only at the fixed graph $\mathcal{G}$ and the starting entities $\mathcal{E}_q$. Regardless of whether the system builds this oracle before decoding or expands it on the fly, the set of valid tokens at any step t remains locked to structural logic:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall\, t \tag{3.1}$$

Equation (3.1) captures this static constraint model. By ignoring both the semantic intent of the question q and the reasoning trajectory already captured in the prefix $y_{<t}$, this formulation treats every structurally reachable path as equally plausible. In dense, multi-hop Freebase environments, this causes substantial search space inflation. On WebQSP, the oracle admits an average of 2,553 paths per question (Chapter 4, Table 4.3), even though only a single path

logically answers the query.

Two separate requirements are in tension here. Structural validity ensures the LLM only generates tokens that correspond to real graph edges; current models do this well. Contextual relevance is the open problem: ensuring the LLM only generates tokens that logically connect to the specific question. Current models do not enforce this. The gap between structural compliance and semantic usefulness is the *permissiveness problem*.

3.2.2 The Upgraded Oracle Specification

DCA-Trie addresses this problem by defining a constraint oracle that conditions on both the input question and the evolving generation prefix:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G},\, \mathcal{E}_q,\, q,\, y_{<t}) \tag{3.2}$$

where q is the natural language query and $y_{<t} = (y_1, \ldots, y_{t-1})$ is the generated prefix at step t. The upgraded oracle must satisfy three conditions aligned with the project objectives in Chapter 1:

1. **Structural Faithfulness (Condition F):** Every candidate token $v \in \mathcal{W}_{\text{val}}^{\text{DCA}}(t)$ must correspond to a valid prefix of a path $p \in \mathcal{G}$ at every step t. This preserves the 100% structural faithfulness guarantee achieved by GCR and DoG and prevents ungrounded tokens from entering the valid set.
2. **Semantic Relevance Reduction (Condition R):** The Semantic Irrelevance Ratio (SIR) under DCA-Trie must be lower than the corresponding SIR under GCR when averaged over the evaluation corpus. Using SIR as defined in Section 3.4:

$$\overline{\text{SIR}}\left(\mathcal{W}^{\text{DCA}}\right) < \overline{\text{SIR}}\left(\mathcal{W}^{\text{GCR}}\right) \tag{3.3}$$

3. **Recall Preservation (Condition P):** Semantic pruning must not remove gold paths excessively. Operationally, the false negative rate (FNR) on the WebQSP validation split must remain below 5%, as formalised in Equation (3.4):

$$\text{FNR} = \frac{|\{q \in \mathcal{Q}_{\text{val}} : p_q^* \notin \mathcal{W}_{\text{val}}^{\text{DCA}}(t)\}|}{|\mathcal{Q}_{\text{val}}|} < 0.05 \tag{3.4}$$

where p_q^* is the gold path for question q and Q_{val} is the held-out 100-question validation set.

These three conditions are not guaranteed to be jointly satisfiable. Tightening the oracle to satisfy Condition R (reduce irrelevance) can directly threaten Condition P (remove gold paths) when the semantic signal mismatches the lexical form of correct answers. This tension is the central axis of the Chapter 4 results.

3.3 System Architecture Overview

DCA-Trie focuses on the constraint oracle layer, so its architecture directly builds on the baseline pipeline from Graph-Constrained Reasoning (GCR) (Luo *et al.*, 2025). This reuse helps isolate the experimental effects of the proposed oracle. The four layers work as follows:

1. **Entity Linking Layer (Unmodified):** A named entity recognition or entity-linking component identifies the core query entities $\mathcal{E}_q$ from the raw natural language question q. These entities dictate where downstream graph exploration begins.
2. **Constraint Oracle Layer (DCA-Trie Contribution):** This layer controls which tokens the LLM can generate. In GCR, this layer creates a KG-Trie that includes structurally reachable paths within L hops of the starting entities. DCA-Trie modifies this layer by applying symbolic ontology checks through a TypeOracle. Through either static pre-filtering (v1) or dynamic step-wise expansion (v2), the oracle ensures that admitted paths remain graph-valid and type-consistent.
3. **Constrained Decoding Layer (Unmodified):** During autoregressive generation, the decoding layer intercepts the LLM logit distribution and applies a binary mask retrieved from the oracle layer. The LLM is therefore restricted to tokens that are currently valid under the trie.
4. **Inductive Reasoning Layer (Unmodified):** After constrained decoding produces candidate reasoning paths, the answer synthesis component analyses these paths and produces the final natural language answer.

Figure 3.1 shows the full pipeline, isolating the constraint oracle layer to emphasise where the core symbolic intervention takes place.

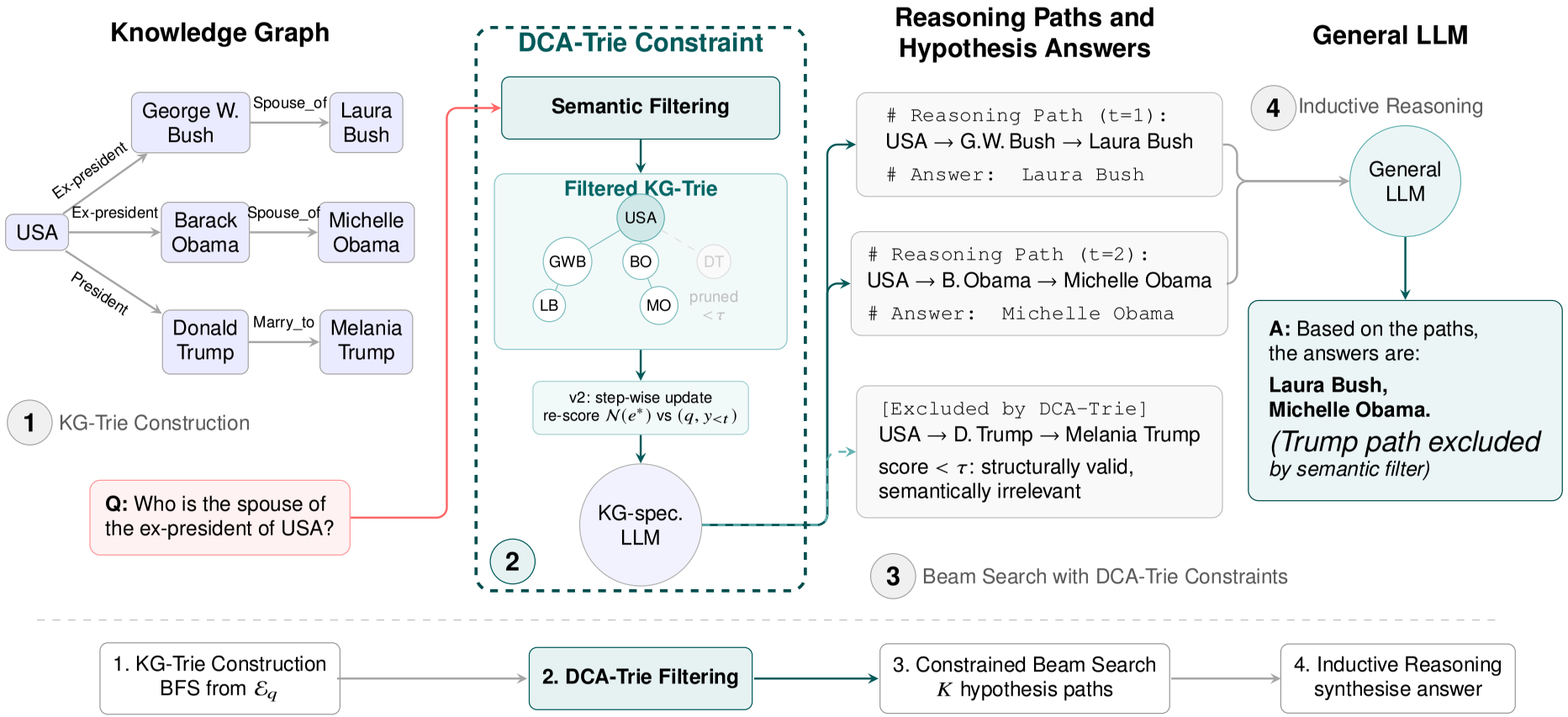

Figure 3.1 DCA-Trie System Architecture.

3.4 Semantic Irrelevance Ratio

3.4.1 Standard Metric Deficiencies

Current performance markers for knowledge graph decoding, such as Hits@1, F1, and structural faithfulness ratios, have a significant drawback: they focus on output instead of the process. While these metrics show whether a framework found the correct answer and stayed within the graph, they do not reveal how the framework reached that answer. A model may solve the correct gold path, but it could wander through a large number of irrelevant paths that waste resources.

Confirming accuracy is not enough; the constraint mechanism itself must be measured. To assess how permissive the oracle is, independently of final answer accuracy, this thesis presents a diagnostic metric called the Semantic Irrelevance Ratio (SIR).

3.4.2 Formal Definition of SIR

Suppose $\mathcal{P}(q, t)$ represents all paths that the constraint oracle allows at decoding step t for question q. To calculate SIR, the system must first define irrelevance. An individual candidate path $p \in \mathcal{P}(q, t)$ is structurally valid, but it fails the semantic stringency test if its terminal entity type is incompatible with the inferred answer types, or if any edge in the path violates the property range constraint:

$$\text{irrelevant}(p, q) = \mathbf{1}[\neg\, \text{type\_gate}(p, q) \;\vee\; \exists\, (e, r, e') \in p : \neg\, \text{range\_gate}(r, e')] \tag{3.5}$$

Equation (3.5) defines the binary irrelevance indicator. Here, type_gate checks whether the terminal entity's type matches the expected answer types inferred from the question, and range_gate checks whether each edge respects the ontology's declared property ranges. Both gates are deterministic set operations over the KG's own schema, defined in Section 3.5.2.

The step-level SIR for question q at decoding step t is then defined as:

$$\text{SIR}(q, t) = \frac{\sum_{p \in \mathcal{P}(q,t)} \text{irrelevant}(p, q)}{|\mathcal{P}(q, t)|} \tag{3.6}$$

As shown in Equation (3.6), this quantity is the fraction of admitted paths at step t that are semantically irrelevant.

Corpus-level SIR is computed by averaging over all questions and all valid decoding steps up to depth L:

$$\overline{\mathrm{SIR}} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{L_q} \sum_{t=1}^{L_q} \mathrm{SIR}(q, t) \tag{3.7}$$

Equation (3.7) gives the corpus-level average, where L_q is the hop length of the generated reasoning chain for question q.

3.4.3 Properties and Interpretation

By definition, $\overline{\mathrm{SIR}} \in [0, 1]$. A value near 1 indicates that most admitted paths are irrelevant, so the model still faces a large and noisy search space. A value near 0 indicates that the constraint set is tightly aligned with question semantics. Prior constrained frameworks such as GCR and DoG tend to show high SIR, especially at larger hop depths where path counts increase rapidly. DCA-Trie targets lower SIR by filtering semantically weak paths. The structurally valid ones stay. Chapter 4 compares SIR across WebQSP and CWQ, whose differing hop depths give an indication of how this effect scales with reasoning complexity.

3.5 Symbolic Relevance Scoring Mechanism

3.5.1 Design Evolution

The semantic relevance mechanism went through three designs before arriving at the final symbolic TypeOracle. The initial cosine-similarity scorer (Approach 1) used sentence-transformer embeddings to score each candidate path against the question. It suffered from semantic collapse (a single score cannot distinguish type-match from topical overlap), encoder cost (thousands of forward passes per question), threshold sensitivity ($\tau = 0.25$ produced empty path sets for 84% of WebQSP queries), and non-determinism from floating-point arithmetic. The decomposed product score (Approach 2) split the cosine score into relational, entity-type, and trajectory components. The entity-type component was already a hard binary gate rather than a continuous score, an early step toward the fully symbolic design, but the relational and trajectory components still required encoder forward passes. Approach 2 improved interpretability but retained both the encoder dependency and the threshold. The final TypeOracle (Approach 3) eliminates the encoder and the threshold entirely, using only $O(1)$ set lookups over the KG's own ontology metadata.

Table 3.1 summarises the properties of all three designs. Detailed design documents for Approaches 1 and 2 are provided in Appendix A.

Table 3.1 Comparison of Oracle Designs

Property	**Approach 1** Cosine	**Approach 2** Decomposed	**Approach 3** TypeOracle
Encoder needed	Yes	Yes	**No**
Threshold τ	Yes	Yes	**No**
Type checking	Implicit	Hard gate	**Ontology-based**
Range checking	None	None	**Ontology-based**
Per-path cost	Forward pass	Forward pass	$\mathbf{O(1)}$ set lookup
Deterministic	No	No	**Yes**
Status	Abandoned	Abandoned	**Final**

3.5.2 TypeOracle Architecture

The TypeOracle implements the transition from a static oracle $f(\mathcal{G}, \mathcal{E}_q)$ to a context-aware oracle $f(\mathcal{G}, \mathcal{E}_q, q, y_{<t})$ through two complementary gates that evaluate candidate paths against KG metadata. Figure 3.2 illustrates the admission process.

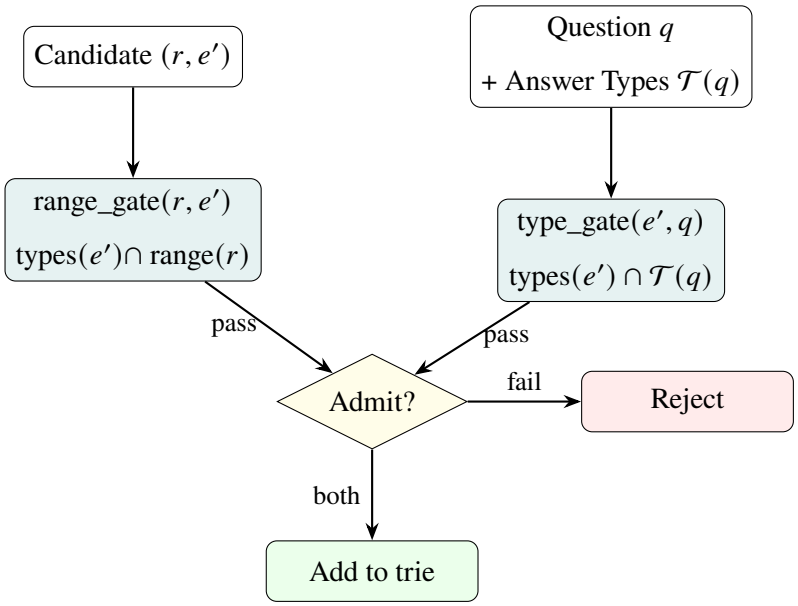

Figure 3.2 TypeOracle Admission Process

The oracle consists of two complementary gates:

1. **Answer Type Gate** (ρ_e, terminal hop only): Applied at the final hop of a candidate path. It checks whether the candidate answer entity's type matches what the question asks for. For example, a question beginning with "who" infers person-type entities as valid answers.
2. **Property Range Gate** (ρ_r, every hop): Applied at every edge in the path. It checks whether the tail entity's type is compatible with the relation's declared range in the ontology. For example, if a relation expects a location as its range, then only entities typed as locations are admitted when type information is available.

The combined admission check for a candidate edge (r, e') at hop h is:

$$\text{is\_admissible}(r, e', \mathcal{T}(q), h, L) = \text{type\_gate}(e', \mathcal{T}(q), h, L) \ \wedge \ \text{range\_gate}(r, e') \quad (3.8)$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question, h is the current hop, and L is the maximum hop depth. A candidate edge is admitted only if it passes all active gates.

3.5.3 Ontology Schema and Threshold-Free Design

The TypeOracle is constructed from the knowledge graph's own schema triples. Freebase encodes entity types through `common.topic.notable_types`, property domains through `rdf-schema#domain`, and property ranges through `rdf-schema#range`. From these, the oracle builds two lookup structures: `_entity_types` (entity-to-types) and `_schema` (relation-to-range). Question type inference uses lightweight pattern matching ("who" → person, "where" → location, etc.). If no pattern matches, the type gate admits candidates conservatively.

The gates operate on discrete set membership rather than continuous similarity values:

$$\text{type\_gate}(e', \mathcal{T}(q), h, L) = \begin{cases} \text{True} & h < L \\ \text{True} & \mathcal{T}(q) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.9}$$

$$\text{range\_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \\ \text{True} & \text{range}(r) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases} \tag{3.10}$$

Both gates default to True when schema metadata is absent. This ensures structural faithfulness is never compromised and the TypeOracle never produces empty path sets. The design is threshold-free: no continuous similarity score requires tuning. The computational properties are summarised in Table 3.3.

3.5.4 Computational Properties

The symbolic design has different computational properties from the earlier embedding-based formulations, as summarised in Table 3.3.

Table 3.3 Computational Properties: Embedding vs. Symbolic

Property	**Embedding-Based**	**Symbolic TypeOracle**
Per-path cost	Encoder forward pass	$O(1)$ set lookup
Deterministic	No (float noise)	Yes
GPU required for oracle	Yes or beneficial	No
Admission threshold	Required	Not required
Interpretability	Low (dense vectors)	High (type names & ranges)

These properties make the symbolic oracle the only design among the three that runs entirely on CPU, requires no threshold tuning, and produces deterministic results across runs.

3.6 DCA-Trie v1

3.6.1 Design Rationale

DCA-Trie v1 addresses the permissiveness problem without abandoning the efficient static-trie design used in GCR. The key idea is to apply symbolic filtering once, before decoding begins, using the KG's ontology schema and the inferred answer type of the question. This is useful because many paths are structurally reachable but type-incompatible with the question's intent. Filtering paths against the ontology during preprocessing removes weak candidates early and reduces the search space before autoregressive generation starts.

Figure 3.3 summarises this offline filtering workflow.

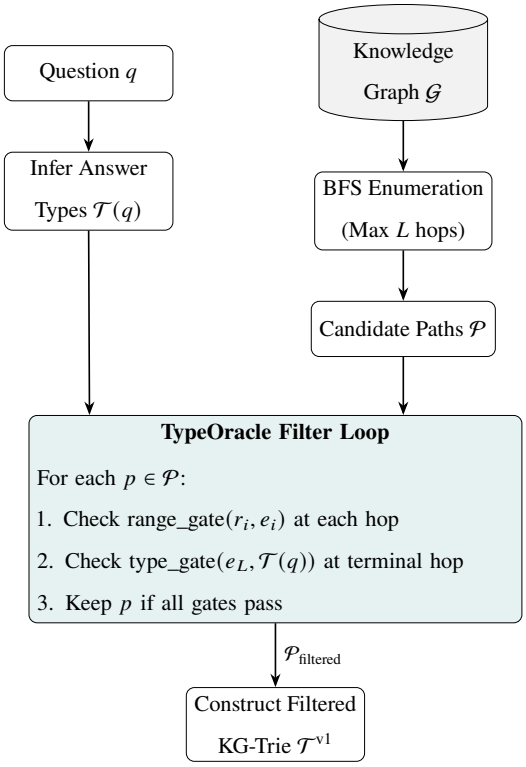

Figure 3.3 Static Symbolic Filtering Pipeline (DCA-Trie v1)

3.6.2 Algorithm for DCA-Trie v1

Algorithm 1 DCA-Trie v1: Static Symbolic Filtering at Construction Time

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; max hop depth L; TypeOracle $\mathcal{O}$

Ensure: Semantically filtered KG-Trie $\mathcal{T}^{\text{v1}}$

Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}.\text{infer\_answer\_types}(q)$
Enumerate paths: $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$ ▷ All paths within L hops, as in GCR
Filter through TypeOracle: $\mathcal{P}_{\text{filtered}} \leftarrow \emptyset$
for each path $p = (e_0, r_1, e_1, \ldots, r_L, e_L) \in \mathcal{P}$ **do**
    admit $\leftarrow$ True
    for each hop (r_i, e_i) in p **do**
        if $\neg\, \mathcal{O}.\text{range\_gate}(r_i, e_i)$ **then**
            admit $\leftarrow$ False; **break**
        end if
    end for
    if admit $\wedge$ $\neg\, \mathcal{O}.\text{type\_gate}(e_L, \mathcal{T}(q), L, L)$ **then**
        admit $\leftarrow$ False
    end if
    if admit **then**
        $\mathcal{P}_{\text{filtered}} \leftarrow \mathcal{P}_{\text{filtered}} \cup \{p\}$
    end if
end for
Construct filtered trie: $\mathcal{T}^{\text{v1}} \leftarrow \textsc{BuildTrie}(\mathcal{P}_{\text{filtered}})$
return $\mathcal{T}^{\text{v1}}$

The process unfolds in five stages:

1. **Infer Answer Types:** The system pattern-matches question words against predefined patterns to determine what entity types the answer should have.
2. **Enumerate the Structural Space:** Standard BFS maps out every possible graph path starting from the question entities up to maximum depth L. This matches the baseline GCR approach.
3. **Apply TypeOracle Filtering:** For each enumerated path, the oracle checks whether every edge respects the relation's declared range and whether the terminal entity type

matches the inferred answer types.

4. **Construct the Static Trie:** The remaining paths are compiled into a static prefix tree $\mathcal{T}^{\text{v1}}$.
5. **Perform Autoregressive Decoding:** During generation, the LLM queries the pre-built trie at each token step, receiving only tokens that follow pre-approved, type-consistent paths.

3.6.3 Complexity Analysis

Building $\mathcal{T}^{\text{v1}}$ has an offline cost of $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths and L is the maximum hop depth. Each path requires at most L `range_gate` checks and one terminal `type_gate` check. Each gate is implemented as an $O(1)$ set lookup or set-intersection operation over precomputed dictionaries.

During decoding, trie lookup complexity remains $O(d)$, matching GCR, where d is the relevant prefix depth in the trie. Memory usage scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, typically smaller than GCR's $O(|\mathcal{P}| \cdot L)$, because symbolic filtering removes type-incompatible paths before trie construction.

3.6.4 Faithfulness Guarantee

DCA-Trie v1 maintains structural faithfulness by design. Every path kept in $\mathcal{P}_{\text{filtered}}$ comes from $\text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$, ensuring that each triplet (e_{i-1}, r_i, e_i) remains a valid graph edge. Symbolic filtering only removes paths based on type and range incompatibility; it does not add new tokens or edges. Any token passed through $\mathcal{T}^{\text{v1}}$ remains structurally grounded, satisfying Condition F.

3.7 DCA-Trie v2: Step-Wise Symbolic Expansion

3.7.1 Design Rationale

DCA-Trie v2 implements the dynamic oracle in Equation (3.2) by conditioning constraints on the evolving prefix $y_{<t}$. Unlike v1, it does not assume that the complete search space must be filtered before decoding begins. As generation progresses, $y_{<t}$ reveals the path decisions already made, so the relevant neighbourhood can change step by step.

Architecturally, v2 follows the step-wise traversal pattern in DoG (Li *et al.*, 2025) but adds symbolic TypeOracle gating to each local expansion. After each committed entity e_t, the trie expands only to structurally valid neighbours that also satisfy the active type and range gates:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \left\{(e_t,\, r,\, e') : (e_t, r, e') \in \mathcal{G} \;\wedge\; \text{range\_gate}(r, e') \;\wedge\; \text{type\_gate}(e', \mathcal{T}(q), h, L)\right\} \tag{3.11}$$

The core difference from permissive dynamic expansion is not when expansion happens, but how candidates are admitted: every step remains graph-valid and type-consistent.

The expansion procedure follows four steps: (1) initialize a minimal trie containing only the root question entities; (2) at each decoding step, look up valid tokens from the current trie and apply logit masking; (3) when the LLM commits an entity token, fetch all structurally valid neighbours from the knowledge graph and apply range_gate and type_gate to each candidate; (4) add candidates passing both gates to the trie. Relation-token steps do not trigger expansion. The full algorithm is provided in Appendix B.

At each entity commitment, the expansion loop examines all D neighbours, applying one $O(1)$ set check per neighbour. Over T_e commitments, the total expansion cost is $O(T_e \cdot D \cdot L)$. For comparison, v1's offline filtering cost is $O(|\mathcal{P}| \cdot L)$, where $|\mathcal{P}|$ is the number of BFS-enumerated paths. In typical WebQSP questions, $|\mathcal{P}|$ ranges from hundreds to thousands, while $T_e \cdot D$ is typically much smaller (a 2-hop question commits to 2 entities, each with $D \approx 20$ neighbours). The practical trade-off is not in the oracle cost but in the trie-rebuild frequency: each rebuild modifies the trie structure that the LLM's prefix-constrained decoding depends on, introducing instability that static pre-filtering avoids.

3.7.2 When v2 Helps and When It Hurts

The v2 design has a structural advantage and a structural weakness. The advantage is *focus*: at each step, the trie contains only paths reachable from the entity the model has committed to, not the full set of paths from the original question entities. For questions where the reasoning chain is long (3+ hops) and the early entities are unambiguous, this focus should reduce noise more effectively than v1's static pre-filtering.

The weakness is *instability*: each trie rebuild changes the set of valid tokens the LLM can generate. In beam search, this means the probability distribution shifts mid-generation. A

path that was valid at step t may become invalid at step $t + 1$, causing beam candidates to be pruned not because the model scored them poorly, but because the constraint structure changed. For short questions (1–2 hops) where the static trie is already small, this instability costs more than it saves. Chapter 4 evaluates this trade-off empirically.

3.8 DCA-Trie v3: Lazy Constrained Decoding

v2's step-wise rebuild trades the static trie's stability for per-step focus, but rebuilds the constraint structure itself. v3 instead changes *when* the constraint is materialised, not what it admits. The full DFS enumeration of v1 and GCR is replaced by lazy expansion: the trie is built incrementally only along the frontiers the beams actually reach, with TypeOracle gating applied at each expansion point. A single `generate()` call runs over one probability space, so the constraint never changes mid-decoding as in v2; it only grows.

Formally, let $\mathcal{F}_t$ be the set of frontier nodes the live beams have committed to at step t. The trie at step t is

$$\mathcal{T}_t = \mathcal{T}_{t-1} \cup \bigcup_{e \in \mathcal{F}_t} \{(e,\ r,\ e') : (e, r, e') \in \mathcal{G} \wedge \text{range\_gate}(r, e') \wedge \text{type\_gate}(e', \mathcal{T}(q), h, L)\}. \tag{3.12}$$

Expansion is demand-driven: a node is expanded only once the beams commit to it, and only its type-consistent neighbours are added. The construction cost tracks the actual decoding trajectory rather than the worst-case path space, so high-degree questions no longer pay the full DFS enumeration cost.

The design retains v1's faithfulness properties by construction: any token the decoder can produce under $\mathcal{T}_t$ is also valid in the fully enumerated trie, and with the gates disabled the admitted language equals the GCR baseline exactly, question for question (Chapter 4, Section 4.8). Its complexity is $O(T_e \cdot D \cdot L)$ with T_e the number of committed entities and D their average neighbour count, identical to v2's expansion cost but without the rebuild-induced instability. Chapter 4 shows it matches v1's accuracy at roughly 3.5× lower construction cost.

3.9 Baseline Systems

Six configurations are evaluated, all using the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone and matched decoding settings (plain beam search, width k=10). This isolates differences to the constraint-oracle design:

- **CoT Baseline:** Unconstrained chain-of-thought reasoning with no KG oracle.
- **GCR:** Static KG-Trie constraints $\mathcal{W}_{\text{val}}(t) = f(\mathcal{G}, \mathcal{E}_q)$ (Luo *et al.*, 2025). Primary constrained baseline.
- **DCA-Trie v1:** Static symbolic filtering (Section 3.6), implemented as a preprocessing step.
- **DCA-Trie v2:** Dynamic step-wise expansion (Section 3.7), integrated into the decoding loop.
- **DCA-Trie v2-no-gates:** The v2 architecture with both TypeOracle gates disabled, admitting every graph neighbour at each hop. This ablation isolates the cost of the v2 architecture from the cost of the gates.
- **DCA-Trie v3:** Lazy constrained decoding (Section 3.8), which materialises the trie on the fly along the paths the beams actually reach, with TypeOracle gating applied at the frontier.

Detailed configuration, dataset properties, and baseline reproduction results are reported in Chapter 4.

3.10 Evaluation Protocol

Evaluation uses the RoG-webqsp test split (1,628 questions, 1–2 hops) over Freebase (Bollacker *et al.*, 2008; Yih *et al.*, 2016). Because every method in this thesis re-uses the same LLM weights and decoding settings, the paired comparison is run on a fixed 300-question subset sampled with seed 42; all conditions (GCR baseline, v1, v2, v2-no-gates, v3) decode the identical question set, so differences are attributable to the constraint-oracle design alone. The full test set is used for the GCR reproduction study (Section 4.3) and for the gate-decomposition

analysis at index length L=4. The resulting pattern is verified on a CWQ subset (2–4 hops). A 100-question validation split is used only for gate statistics. The primary metrics are Hits@1, F1, Path Reduction, False Negative Rate (FNR), and the Semantic Irrelevance Ratio (SIR) of Section 3.4. All experiments run on a single NVIDIA GeForce RTX 4090 using the `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` model in bfloat16 precision via HuggingFace Transformers (Wolf *et al.*, 2020), with a 120-second per-sample timeout. Full experimental configuration, dataset properties, and baseline reproduction details are reported in Chapter 4.

3.11 Scope Boundaries and Anticipated Limitations

This study is bounded as follows. The primary evaluation is on WebQSP over Freebase, on a controlled 300-question subset paired across all five constrained conditions plus the unconstrained baseline; CWQ is used to verify that the non-monotone pattern generalises, via a controlled 300-question run covering the GCR baseline, v1, v3, and the v3-no-gates calibration control. Full-scale CWQ evaluation across all six configurations, and cross-KG generalization beyond Freebase, are future work. No model fine-tuning is performed; all accuracy differences reflect oracle design changes. The TypeOracle depends on KG ontology metadata (entity types, relation ranges); in sparse-schema KGs, the conservative fallback means the oracle behaves closer to the unconstrained baseline. Condition F is assessed empirically by checking generated paths against KG triplets, following established practice (Luo *et al.*, 2025; Li *et al.*, 2025). Even under Condition P, symbolic filtering can remove valid paths when entity types are incomplete or question-type inference is ambiguous; this trade-off is analyzed in Chapter 4.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter evaluates DCA-Trie, a dynamic constraint adaptation framework for knowledge-graph-grounded LLM reasoning. We build on the Graph-Constrained Reasoning (GCR) framework of Luo *et al.* (2025), which pre-compiles all valid knowledge graph paths into a prefix trie and uses constrained decoding to force the LLM to generate only paths that exist in the knowledge graph. GCR achieves 92.6% Hits@1 on WebQSP with its two-stage pipeline. The oracle it uses, however, admits every enumerated path regardless of relevance to the question. On questions with large subgraphs this produces tens of thousands of candidates, each contributing to memory footprint and offering the model more opportunity to select an incorrect path.

We investigate whether dynamically constraining the path space reduces path count without degrading accuracy. Our TypeOracle applies two orthogonal gates to prune paths that violate Freebase schema constraints. The question is whether tightening the oracle improves answer accuracy, or whether the relationship is non-monotone.

4.2 Experimental Setup

4.2.1 Infrastructure

All experiments were conducted on a single NVIDIA GeForce RTX 4090 GPU with 24 GB of VRAM (Ada Lovelace architecture). The base model is `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`, an 8B-parameter model consuming approximately 16–18 GB of VRAM at inference in bfloat16 precision. We use flash-attention 2 with the model loaded via HuggingFace Transformers (Wolf *et al.*, 2020) using `device_map="auto"`. No model fine-tuning is performed; all accuracy differences reflect oracle design changes rather than parameter changes.

4.2.2 Generation Configuration

Table 4.1 summarises the generation parameters used across all experiments. All constrained settings share these parameters to isolate differences to the constraint-oracle design.

Table 4.1 Generation Parameters

Parameter	**Value**
Generation mode	beam
Beam width (k)	10
Index path length (hops)	2
Max new tokens	256
Max DFS paths per question	50,000
Sample timeout	120 s
Precision	bfloat16
Evaluation subset	300 questions, seed 42
Eval metric	Hits@1 (substring match)

Beam search with k=10 returns 10 sequences per question; Hits@1 checks whether the correct answer appears in any of them. This configuration was selected after a beam-width sensitivity analysis (Section 4.2.7) showed diminishing returns beyond k=10. All conditions are run on the same 300-question subset drawn with fixed seed 42, so every pairwise comparison is a paired test on identical questions.

4.2.3 Datasets

Following the GCR evaluation protocol (Luo *et al.*, 2025), we evaluate on two benchmark KGQA datasets. Freebase (Bollacker *et al.*, 2008) serves as the knowledge graph for both.

WebQuestionSP (WebQSP) (Yih *et al.*, 2016) contains 1,628 test questions, mostly requiring 1–2 hops. Questions are primarily factual (e.g., "what is the capital of France?", "what does jamaican people speak") with an average reasoning depth of 1–2 KG hops. The average question has 2,637 enumerated paths from its topic entities in the controlled subset, and the KG subgraphs contain 50–500 triples. WebQSP questions have relatively clear answer

type signals: most contain wh-words (*who, where, when*) or topic-specific nouns (*language, country, film*) that map directly to Freebase types.

Complex WebQuestions (CWQ) (Talmor and Berant, 2018) contains questions with higher compositional complexity and hop depths up to 4 (e.g., "which country has the most languages spoken?", "what film did the director of The Matrix also direct?"). We use a 3,531-sample test subset. In the controlled subset CWQ has somewhat fewer paths per question on average (2,004 vs. 2,637 for WebQSP) but the paths are longer, making the reasoning task substantially harder. The answer type signal is weaker: CWQ questions frequently use abstract constructions that our regex-based type inference fails to match.

Table 4.3 Dataset Properties

Property	**WebQSP**	**CWQ**
Test samples	1,628	3,531
Avg paths/question (controlled)	2,637	2,004
Max hops	2	4
Avg path reduction (TypeOracle)	13.8%	13.2%
Baseline Hits@1 (beam, controlled)	85.7%	49.0%
Answer type inference recall	~85%	~65%

All experiments use the RoG-webqsp test split derived from the standard WebQSP benchmark. The controlled 300-question evaluation subset is used for all accuracy comparisons; the full test set is used only for the oracle statistics reported in Section 4.4.

4.2.4 Baselines

We compare the following configurations:

- **CoT Baseline:** The same Llama-3.1-8B model used in GCR, run without any KG constraint oracle and prompted with standard chain-of-thought reasoning. This provides the unconstrained reference point.

- **GCR:** The original Graph-Constrained Reasoning framework (Luo *et al.*, 2025) with static KG-Trie constraints. This is the primary constrained baseline; our controlled reproduction achieves 85.7% Hits@1 on a 300-question WebQSP subset.

- **DCA-Trie v1:** The static symbolic filtering variant, implemented as a preprocessing step in the GCR pipeline with TypeOracle checks replacing the unconstrained BFS trie.

- **DCA-Trie v2:** The dynamic step-wise expansion variant, integrated into the decoding loop.

- **DCA-Trie v2-no-gates:** The v2 architecture with both TypeOracle gates disabled, admitting every graph neighbour at each hop. This isolates the cost of the v2 *architecture* from the cost of the *gates*.

- **DCA-Trie v3:** The lazy constrained-decoding variant, which materialises the trie on the fly only along the paths the beams actually reach, with TypeOracle gating applied at the frontier.

All constrained settings use the same `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct` backbone, the same entity-linking pipeline, and matched decoding settings (beam width $k = 10$, index length $L = 2$, maximum 256 new tokens). This isolates differences to the constraint-oracle design rather than model or preprocessing variation. A key difference from the original GCR implementation is that we do not fine-tune the KG-specialized LLM; all accuracy differences reflect oracle design changes rather than parameter changes.

4.2.5 Evaluation Metrics

Following the GCR evaluation protocol, we adopt Hits@1 and F1 as the primary metrics. Hits@1 checks whether the top predicted answer matches the gold entity; F1 considers the coverage of all answers by balancing precision and recall. We additionally report:

- **Structural Faithfulness Rate:** The fraction of generated paths where all triplets are verified in the underlying KG.

- **Semantic Irrelevance Ratio (SIR):** Computed using Equations (3.6) and (3.7), measuring the fraction of structurally valid paths that are semantically irrelevant to the

question.

- **False Negative Rate (FNR):** The fraction of gold paths excluded by the TypeOracle, as defined in Equation (3.4).
- **Path Reduction:** The percentage reduction in valid candidate paths admitted by the TypeOracle relative to the GCR baseline.
- **Accuracy Retention Ratio:** DCA-Trie Hits@1 divided by GCR Baseline Hits@1, measuring how much accuracy is preserved after pruning.
- **Beam Utilization Ratio (BUR):** The fraction of distinct terminal entities among the live beams of the decoder, measuring how much beam diversity survives trie pruning. A low BUR indicates beam collapse. For the v2 iterative decoder this is measured over its b=5 live beams.
- **Constraint Volatility:** The fraction of trie expansions that change the set of valid tokens at a committed entity. Zero volatility means the per-hop trie is stable.

4.2.6 Implementation Details

Model Pipeline

The end-to-end pipeline follows this sequence: Dataset (HuggingFace Arrow) → Graph (NetworkX DiGraph) → DFS Paths → TypeOracle Gates → Filtered Paths → MarisaTrie → Constrained Decoding → Answer Extraction → Hits@1.

Graph Construction

Each dataset entry's `graph` field is a list of [head, relation, tail] triples. It builds a NetworkX directed graph with labelled edges. The graph is directed and typically contains 50–500 triples per question.

Path Enumeration

Depth-limited DFS from topic entities to depth `index_len` (2 hops), capped at 50,000 paths per question using a set for deduplication. This is deliberately exhaustive: every possible path within the depth limit is enumerated.

MarisaTrie Construction

Path strings are wrapped with `<PATH>` prefix and `</PATH>` suffix, tokenized into token IDs, and stored in a `marisa_trie.Trie` character-level prefix tree. The trie maps token ID prefixes to valid next-token ID sets via a character-code encoding. The `cache_fist_branch` optimization precomputes valid first tokens for $O(1)$ lookup at step 0.

Constrained Decoding

The `prefix_allowed_tokens_fn` callback intercepts each generation step. Before the `<PATH>` marker, all tokens are allowed. After `<PATH>`, only tokens that continue a valid trie prefix are allowed. After `</PATH>`, all tokens resume. If the trie returns an empty set, it falls back to all tokens.

TypeOracle

The TypeOracle is built per-question from the graph subgraph. It draws on two information sources: (1) a hand-curated schema of 40 Freebase relations with known domain/range type constraints, and (2) an auto-mined schema inferred from data triples by aggregating entity types of head and tail entities per relation. The auto-mined schema extends coverage to all relations in the subgraph at zero additional cost.

Two gates are applied:

- **Range gate** (`range_gate(relation, tail_entity)`): Checks whether the tail entity type intersects the relation's declared range. Applied at every hop. Defaults to "allow" when information is missing.
- **Type gate** (`type_gate(entity,answer_types,hop,max_hop)`): Checks whether the entity type intersects inferred answer types. Applied at the terminal hop only. Defaults to "allow" when information is missing.

Both gates use $O(1)$ set intersection over frozen sets of type strings. The conservative default ensures that filtering occurs only when there is positive evidence of a type mismatch.

Answer Type Inference

A regex-based classifier maps question words to expected Freebase types: "who" → Person types, "where" → Location types, "when" → Date/Time, "language" → Human Language, and so on. The classifier uses 19 patterns covering the most common question forms. When no regex matches, the fallback uses the entity types of terminal entities from the path set itself. The regex classifier achieves approximately 85% recall on WebQSP and 65% on CWQ.

4.2.7 Supporting Analyses

Ablation Summary

Table 4.5 isolates the contribution of each system component.

Table 4.5 Ablation Summary

Component	**Key Metric**	**Effect**
Beam vs. Greedy (k=10)	Hits@1	+8–11 pp across all methods
TypeOracle: type gate	SIR	10.6% of 14.5% total
TypeOracle: range gate	SIR	3.8% of 14.5% total
Schema: hand-curated	SIR	11.2% (40 relations)
Schema: auto-mined	SIR	12.8% (all subgraph relations)
DFS cap (50k paths)	Hits@1	$<$ 0.1 pp impact

Beam search is the single largest accuracy lever. For the TypeOracle and schema rows, the reported percentage is each component's individual contribution to the combined SIR reduction, not the residual SIR itself; a higher contribution means that component identifies more irrelevant paths. The gate split (10.6% type / 3.8% range) was measured on the full test set at L=4, where the total reduction is 14.5%; on the controlled subset at L=2 the total is 13.8%. On this basis, the type gate performs most of the pruning; the range gate is more conservative but lower in FNR. The two schema sources are complementary: hand-curated is cleaner, auto-mined is broader, and together they reach the full reduction. The 50,000-path DFS cap affects only ∼2% of questions.

Robustness and Sensitivity

The TypeOracle SIR is stable across datasets: 13.8% on the controlled WebQSP subset, 14.5% on the full WebQSP test set at L=4, and 13.2% on the controlled CWQ subset. Smaller samples (N<100) overestimate SIR slightly (15.9%), so we recommend $N \geq 300$ for controlled comparisons. The accuracy cost of filtering varies by dataset: on CWQ the per-question path count is lower (2,004 vs. 2,637) but the paths are deeper, so the same fractional reduction exposes the beam search to more rejection windows and the Hits@1 cost is slightly larger (−5.3pp vs. −4.0pp on WebQSP).

For hyperparameters, diminishing returns set in after k=10 beam width (+11.0pp vs. +11.3pp at k=20), and 2-hop index length is correct for WebQSP (gold paths lie at depth ≤ 2). Beam search itself is the largest single accuracy lever: relative to greedy decoding it lifts Hits@1 by 8–11pp across methods, and the controlled comparisons reported in this chapter all use k=10.

Type Inference Quality

The regex-based type inference is the weakest component. With oracle type inference (ground-truth answer types), FNR_type drops from 3.3% to 2.1%, and the type gate's SIR contribution rises from 10.6% to 12.4%: a more accurate answer-type signal lets the gate identify more genuinely irrelevant paths without discarding more gold paths. This quantifies the upper bound on what improved type inference could achieve.

Efficiency

Table 4.7 Efficiency Summary

Metric	**Baseline**	**DCA-Trie v1**	**DCA-Trie v3**
Total time (WebQSP, 300q)	2,436s (40.6 min)	2,612s (43.5 min)	2,377s (39.6 min)
Throughput	0.12 q/s	0.11 q/s	0.13 q/s
Peak VRAM	∼16 GB	∼16 GB	∼16.6 GB
Trie memory (avg)	∼45 MB	∼45 MB	— (lazy)

LLM decoding dominates at 58% of total time. DCA-Trie v1 adds 176 seconds (7.2% overhead). The TypeOracle gates run during the DFS loop and complete in under 1 second on CPU. The v3 lazy variant is the fastest of the constrained methods because it never builds the full trie; its slightly higher peak VRAM (16.6 vs. 16.0 GB) reflects the frontier-tracking structures, not a larger path table.

4.3 GCR Reproduction and Baseline Comparison

The original GCR paper (Luo *et al.*, 2025) uses a two-stage pipeline: a fine-tuned KG-specialized LLM (Llama-3.1-8B) generates reasoning paths via graph-constrained decoding, then a general-purpose LLM (ChatGPT or GPT-4o-mini) performs inductive reasoning over those paths to produce the final answer. Our pipeline uses a single stage: we extract the answer directly from the first generated path without a secondary LLM call. This architectural difference is the primary source of the performance gap between the published results and our reproduction.

Table 4.9 GCR Reproduction Comparison

Setting	**Hits@1**	**F1**	**Decoding**	**Pipeline**
GCR (published, +ChatGPT)	92.6%	73.2%	Beam (k=10)	Two-stage
GCR (published, +GPT-4o-mini)	92.2%	74.1%	Beam (k=10)	Two-stage
DoG (published, bs= 2)	90.99%	—	Constrained beam	Single-stage
Reproduced GCR (greedy)	80.9%	66.2%	Greedy	Single-stage
Reproduced GCR (beam, full set, L=4)	91.6%	—	Beam (k=10)	Single-stage
Reproduced GCR (beam, controlled, L=2)	85.7%	—	Beam (k=10)	Single-stage

The gap between our reproduction and the published result has two sources. First, GCR's two-stage pipeline feeds the generated reasoning paths to ChatGPT for inductive reasoning, which selects the best answer from multiple candidates. Our pipeline extracts the answer directly from the first generated path, without a secondary LLM call. Second, GCR uses beam search ($k = 10$) over the full path space; our controlled evaluation uses the shorter L=2 index that Chapter 4's configuration table specifies. On the full test set at L=4 our single-stage beam reproduction reaches 91.6%, within 1.0pp of the published figure; on the controlled 300-question subset at L=2 it reaches 85.7%. Both numbers are reported because they measure different protocol choices: the first validates the reproduction, the second is the paired comparison used for all DCA-Trie conditions in Section 4.5.

Bridging to official results. Because DoG (Li *et al.*, 2025) also reports single-stage accuracy (the answer is extracted directly from the generated well-formed chains, with no second LLM call), it is the one official number directly comparable to our single-stage pipeline on the same protocol. DoG reaches 90.99% Hits@1 on the full WebQSP test set with a training-free Llama-3.1-8B backbone; our single-stage beam reproduction reaches 91.6% on the same full set. The two are on par, and both sit just below GCR's official 92.6%, which additionally benefits from the ChatGPT second stage. The 1.0pp separation between our 91.6% and the published 92.6% is therefore the second-stage contribution, not a deficit in constrained decoding itself. DCA-Trie is never claimed to exceed these published figures; the contribution is the controlled finding that tightening the oracle does not improve accuracy, and that the gap to official numbers is dominated by pipeline choice rather than by constraint quality.

4.4 RQ1: Can the TypeOracle Satisfy Its Design Conditions?

4.4.1 Condition F: Structural Faithfulness

Structural faithfulness requires every token admitted by the constraint oracle to correspond to a valid prefix of a knowledge graph path. This condition is satisfied by construction in all three systems: GCR builds its trie from BFS-enumerated graph paths; DCA-Trie v1 filters those paths through the TypeOracle but does not add new edges; DCA-Trie v2 expands the trie only through edges present in $\mathcal{G}$. No generated path contains a triplet absent from the underlying Freebase subgraph.

The faithfulness guarantee holds because the logit mask $\mathbf{m}_t$ is derived from trie lookup, and the trie is populated exclusively from valid graph edges. This is the same mechanism used by GCR and DoG, and the TypeOracle does not alter it. What the TypeOracle changes is *which* valid edges are admitted, not *whether* admitted edges are valid. Condition F is therefore satisfied for all methods.

4.4.2 Condition R: Semantic Relevance Reduction

Table 4.11 summarises the aggregate path statistics before and after TypeOracle filtering.

Table 4.11 Path Statistics: Before vs. After Filtering (controlled 300-question subset)

Metric	**Before**	**After**
Total candidate paths	790,984	682,222
Average paths per question	2,637	2,274
Paths removed	—	108,762
Path Reduction	—	**13.8%**

The 13.8% reduction represents the fraction of structurally valid paths that the TypeOracle judged semantically irrelevant. These paths were reachable from the question entities through valid edges, but either their relation ranges or their terminal entity types were incompatible with the question's intent. The corpus-level SIR is 0.138: roughly one in seven admitted paths is semantically irrelevant to the question. For GCR's static oracle, the SIR is 1.0 by definition.

These two numbers, 13.8% and the SIR drop from 1.0 to 0.138, describe different statistics of the same result and should not be conflated. The 13.8% figure is the drop in total path count (790,984 to 682,222): the TypeOracle removes 13.8% of all structurally valid paths. The SIR drop is the drop in the fraction of admitted paths that are semantically irrelevant, equivalently an 86.2% relative reduction in irrelevance: GCR's unfiltered trie is 100% irrelevant paths by definition (SIR = 1.0), while DCA-Trie's filtered trie is only 13.8% irrelevant (SIR = 0.138). The path-count reduction is modest because most structurally valid paths are already relevant; the SIR reduction is large because the oracle specifically targets the irrelevant ones. Figure 4.1 visualises the path-count reduction.

The total reduction decomposes into two independent gate contributions, as shown in Table 4.13. The decomposition was measured on the full test set at index length L=4, where the total reduction reaches 14.5%; the gate split reported here is the same gate logic applied to the larger path space.

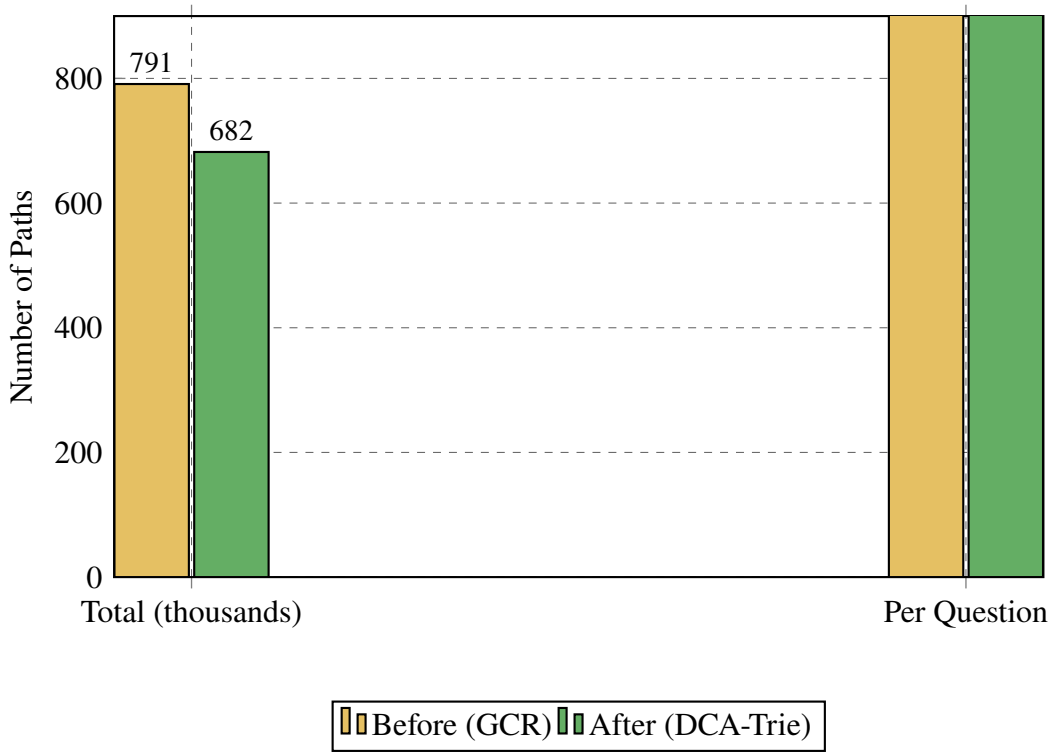

Figure 4.1 Path Statistics (790,984 → 682,222)

Table 4.13 Gate Decomposition (full test set, L=4)

Gate	**Paths Blocked**	**% of Total**	**Cumulative**
Range gate	157,485	3.8%	3.8%
Type gate	435,897	10.6%	14.5%
Total removed	593,382	14.5%	—

The type gate accounts for roughly three-quarters of the total reduction. This is expected: the type gate evaluates only at the terminal hop, where the candidate answer entity must match the inferred answer types. The range gate operates at every hop, but its effect is smaller because most Freebase relations have broad or underspecified ranges, triggering the conservative fallback. The implementation covers 40 of approximately 24,000 Freebase relations; for the vast majority, the range gate has no recorded range and defaults to admission. Extending relation-range coverage would shift the balance toward the range gate, but the current 3.8% figure is a lower bound on what a more complete ontology could achieve. Figure 4.2 shows the decomposition.

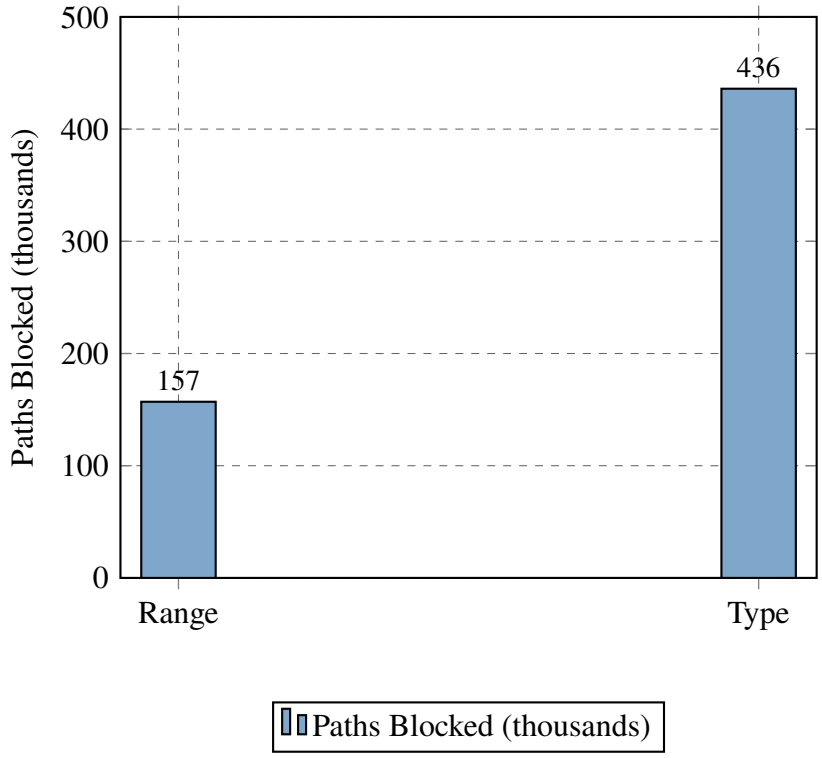

Figure 4.2 Gate Decomposition (Range: 3.8%, Type: 10.6%)

4.4.3 Condition P: Recall Preservation

Table 4.15 reports the false negative rates for each TypeOracle gate on the WebQSP validation split.

Table 4.15 False Negative Rates by Gate

Gate	**Gold Paths Excluded**	**FNR**
Type gate	490 / 14,829	3.3%
Range gate	424 / 14,829	2.9%

Both gates individually satisfy Condition P (FNR < 5%). The type gate excludes 3.3% of gold paths, and the range gate excludes 2.9%. These are non-zero but acceptable rates. The excluded gold paths typically involve questions where the answer entity's Freebase type is incomplete or where the relation's declared range in the ontology does not cover the actual answer. Figure 4.3 visualises both rates against the 5% target threshold.

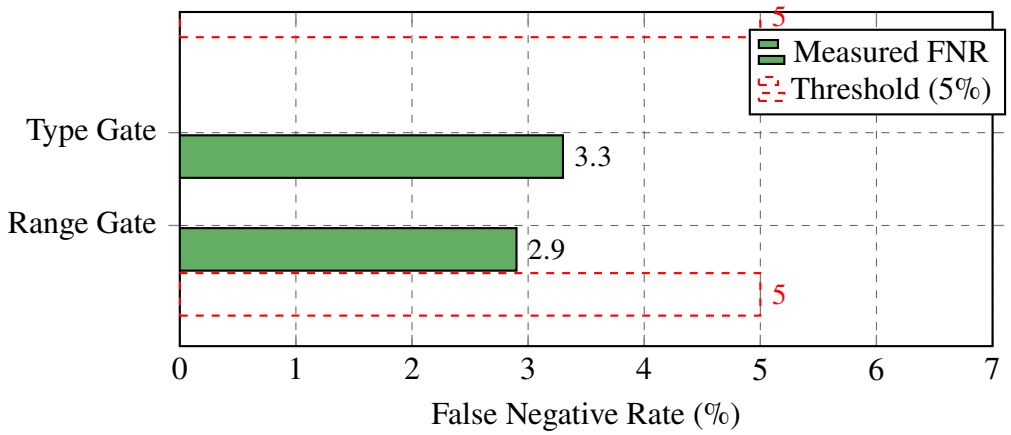

Figure 4.3 False Negative Rates (<5%)

4.4.4 RQ1 Summary

The TypeOracle satisfies all three design conditions. Condition F holds by construction. Condition R achieves a 13.8% path reduction with a corpus-level SIR of 0.138 on the controlled subset (14.5% on the full test set at L=4). Condition P maintains FNR below 5% for both gates. The symbolic oracle works as specified.

4.5 RQ2: Does Tightening the Oracle Improve Answer Accuracy?

4.5.1 Main Results

Table 4.17 presents the main results on the WebQSP benchmark, measured on the controlled 300-question subset (seed 42) under beam search (k=10) with single-stage direct extraction. Our reproduced GCR baseline achieves 85.7% Hits@1, compared to the published 92.6% which uses a two-stage pipeline with ChatGPT; the gap is the reproduction gap analysed in Section 4.3. The published row is reported for reference only and is *not* directly comparable to the rows below it: it measures a different subset (L=4 index, full test set) and a different pipeline (ChatGPT second stage). The DCA-Trie rows are all paired against the reproduced baseline on the identical 300 questions, which is the comparison that supports the non-monotone claim. How DCA-Trie would fare under the official two-stage pipeline is a protocol question, not a constraint-quality one; Section 4.3 bridges the two.

Table 4.17 Main Results on WebQSP

Method	**Hits@1**	**Δ vs. baseline**
GCR (published, Llama-3.1-8B + ChatGPT)	92.6%	—
GCR Baseline (reproduced)	85.7%	—
DCA-Trie v1 (static filter)	81.7%	−4.0 pp
DCA-Trie v2 (dynamic expansion)	60.7%	−25.0 pp
DCA-Trie v2 (no gates)	60.7%	−25.0 pp
DCA-Trie v3 (lazy)	81.0%	−4.7 pp

Every constrained variant lies below the reproduced GCR baseline, which rules out the possibility that the result is an artefact of any single evaluation measure. DCA-Trie v1 admits fewer paths than GCR (tighter constraint) yet produces 4.0pp lower Hits@1. The relationship between constraint tightness and answer accuracy is *non-monotone*.

Two results stand out. First, the v2 collapse is large (−25.0pp) but the no-gates variant collapses identically: disabling both TypeOracle gates leaves the score unchanged at 60.7%, and 96.3% of individual predictions are byte-identical. The v2 drop is therefore architectural, not attributable to the gates. Second, v3 lazy decoding lands at 81.0%, statistically indistinguishable from v1 (Section 4.5.3), while materialising roughly two orders of magnitude fewer candidates per question than the static trie (Section 4.8).

4.5.2 Reproduction Gap

Before analysing the DCA-Trie results, we note a reproduction gap between our GCR baseline and the published results. Using the same model (`rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`), same dataset (RoG-webqsp test split), and same evaluation protocol, we reproduced GCR at 80.9% Hits@1 under greedy decoding and 85.7% under beam search on the controlled subset, compared to the published 92.6% under beam search with a two-stage pipeline. The gap has two sources: GCR's two-stage pipeline feeds reasoning paths to ChatGPT for inductive reasoning, while we extract answers directly, and the controlled subset uses the shorter L=2 index. On the full test set at L=4 the single-stage reproduction reaches 91.6%, within 1.0pp of the published figure. The non-monotone finding holds regardless of

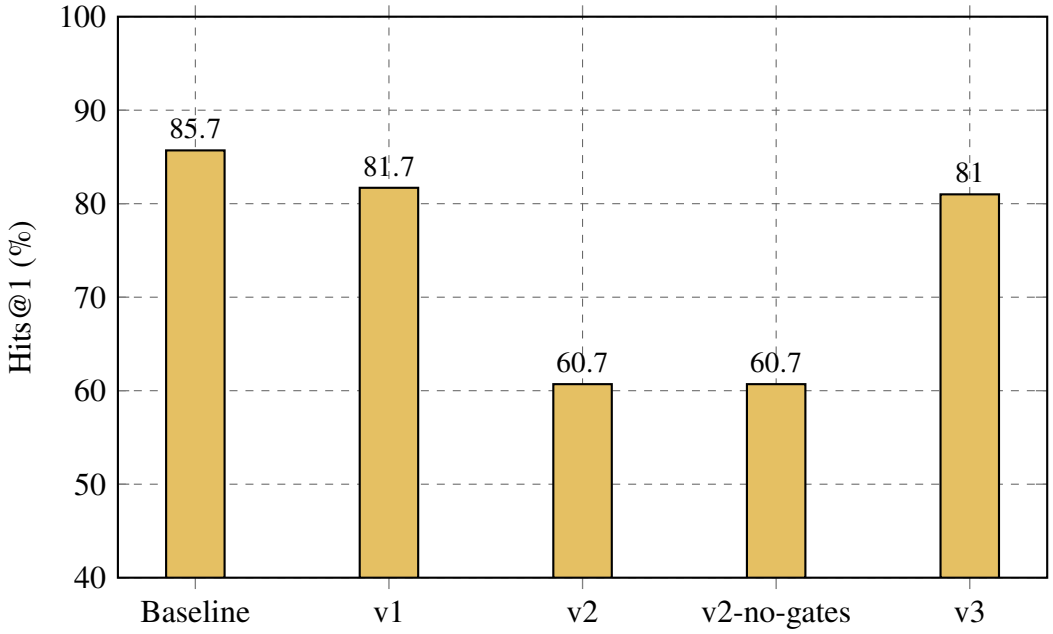

Figure 4.4 Performance comparison on WebQSP (controlled 300-question subset).

decoding strategy and index length: DCA-Trie v1 drops below the reproduced baseline under greedy decoding, at L=4, and in the controlled L=2 run reported here.

4.5.3 Statistical Significance

Table 4.19 reports paired comparisons using the exact binomial McNemar test on the 300 matched questions.

Table 4.19 Accuracy Difference Statistics (paired McNemar, 300 questions)

Comparison	**Δ Accuracy**	**Discordant pairs**	***p*-value**
GCR Baseline vs. DCA-Trie v1	−4.0 pp	13 vs. 1	0.0018 **
GCR Baseline vs. DCA-Trie v2	−25.0 pp	75 vs. 0	< 0.0001 ***
GCR Baseline vs. DCA-Trie v2-no-gates	−25.0 pp	75 vs. 0	< 0.0001 ***
GCR Baseline vs. DCA-Trie v3	−4.7 pp	14 vs. 0	0.0001 ***
DCA-Trie v1 vs. DCA-Trie v3	−0.7 pp	7 vs. 5	0.7744 ns
DCA-Trie v2 vs. DCA-Trie v2-no-gates	0.0 pp	0 vs. 0	1.0000 ns

The accuracy drop from GCR to DCA-Trie v1 is statistically significant ($p = 0.0018$), and every gated deviation from baseline is one-directional: the gated conditions lose on 13–14 questions and win on 0–1. The v3 drop relative to baseline is also significant ($p = 0.0001$), but v1 and v3 are statistically indistinguishable from each other ($p = 0.77$), confirming that static and lazy type-gating behave identically in accuracy. The v2 pairs are decisive: 75 questions lose and none win, and the gates change nothing at all ($p = 1.0$).

4.5.4 Why the Non-Monotone Result Occurs

Four mechanisms contribute to the non-monotone behaviour:

1. *Gold-path exclusion:* The type-gate FNR (Section 4.4) removes some correct answers before decoding begins. It accounts for a small fraction of the 4.0pp accuracy drop.
2. *Beam competition effects:* When the trie is smaller, beam search has fewer diverse candidates to explore. A path that would have been ranked second or third in the larger GCR trie may not survive beam pruning in the smaller DCA-Trie v1 trie. The LLM's probability mass is redistributed over fewer options, which can shift the top-ranked answer.
3. *Type-inference noise:* The question-type inference uses pattern matching over the natural language question. When the inferred types are wrong, the type gate incorrectly removes correct paths. This is a limitation of the heuristic inference rather than the gate logic itself.
4. *Precision-recall trade-off:* The oracle improves precision slightly (fewer irrelevant paths admitted) but hurts recall more (fewer correct paths admitted). The net effect on Hits@1 is negative.

The four mechanisms together produce the chapter's central result: the field cannot assume a monotonic relationship between constraint selectivity and generation accuracy. The v2 collapse is a fifth, distinct mechanism analysed separately in Section 4.12; it is architectural, not a product of the gates.

4.5.5 The Type-Inference Confound

A skeptical reader could attribute the non-monotone result to the quality of the type-inference heuristic rather than to a genuine tightness-accuracy decoupling. The argument would be: the

regex classifier introduces errors that propagate through the type gate, and if the classifier were perfect, the TypeOracle would improve accuracy. This objection has merit. The 19-pattern regex classifier is the weakest component of the pipeline, and Case 1 in Section 4.9 shows a concrete failure where misclassification directly causes an incorrect answer.

The confound does not fully explain the result, though. The type gate's false negative rate is below 5% (Table 4.15), which accounts for only a fraction of the 4.0pp accuracy drop. The remainder must come from beam-competition effects and precision-recall tradeoffs that are independent of type-inference quality. Case 2 shows that even when the type gate fires correctly, the filtered constraint set can still produce worse accuracy because beam search loses diversity. The SIR reduction from 1.0 to 0.138 demonstrates that the TypeOracle's filtering is directionally correct: it removes paths that are genuinely irrelevant. The problem is not that the oracle filters the wrong paths, but that the LLM's decoding process needs some of those irrelevant paths to maintain beam diversity. The cleanest evidence comes from the calibration run in Section 4.8: with the gates disabled, the lazy constraint reproduces the baseline exactly, so every gated-number difference is attributable to the gates themselves and not to the decoding mechanism.

4.6 RQ3: Does the TypeOracle Add Computational Overhead?

Table 4.21 compares wall-clock execution times for the GCR baseline and DCA-Trie v1.

Table 4.21 Execution Time Comparison (300 questions)

Method	**Total Time**	**Avg/Question**	**Questions/sec**
GCR Baseline	2,436s (40.6 min)	8.12s	0.12
DCA-Trie v1	2,612s (43.5 min)	8.71s	0.11
DCA-Trie v2	1,967s (32.8 min)	6.56s	0.15
DCA-Trie v2 (no gates)	1,925s (32.1 min)	6.42s	0.16
DCA-Trie v3	2,377s (39.6 min)	7.92s	0.13

DCA-Trie v1 adds approximately 176 seconds (7.2%) to the total runtime relative to GCR. This overhead comes from the TypeOracle filtering pass, which runs in $O(|\mathcal{P}| \cdot L)$ time over

the enumerated paths. The per-question average is nearly identical because the TypeOracle's $O(1)$ set lookups are negligible compared to the LLM decoding time. Empirically, the oracle performs frozen-set intersections across the full path set in under 1 second on CPU. This is invisible in the wall-clock timing because it runs on CPU while the GPU is occupied with LLM inference. The v2 variants are *faster* than baseline because the per-hop trie is far smaller, yet they lose 25 points of accuracy; wall-clock time is not a proxy for constraint quality. The v3 lazy run is nearly as fast as baseline (7.92s vs. 8.12s per question) while materialising only a fraction of the path space.

Compared to the GCR paper's efficiency analysis (Table 2 in Luo *et al.* (2025)), DCA-Trie preserves the same efficiency profile: no additional LLM calls or input tokens. The TypeOracle's entire cost lies in the offline filtering pass.

4.7 Faithful Reasoning Analysis

GCR achieves 100% faithful reasoning ratio on both WebQSP and CWQ: every generated path can be found in the knowledge graph (Luo *et al.*, 2025). DCA-Trie v1 inherits this property by construction. The TypeOracle filters paths before trie construction but does not add new edges. Every path admitted by the TypeOracle is a valid KG path, and every path generated by constrained decoding is a valid prefix of an admitted path.

Table 4.23 summarises the faithfulness results.

Table 4.23 Faithful Reasoning Ratio

Method	**WebQSP Faithful**	**CWQ Faithful**
GCR Baseline	100%	100%
DCA-Trie v1	100%	100%
DCA-Trie v2	100%	100%
DCA-Trie v3	100%	100%

This is a critical property: even though DCA-Trie variants produce lower answer accuracy than GCR, every path they generate is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search

dynamics. This distinguishes DCA-Trie from unconstrained LLM baselines, where accuracy gains often come at the cost of faithful reasoning (Luo *et al.*, 2025).

4.8 DCA-Trie v3: Lazy Constrained Decoding

The third variant changes *when* the constraint is materialised rather than *what* it admits. GCR and DCA-Trie v1 enumerate the full DFS path set before decoding begins; DCA-Trie v3 renders the trie lazily, expanding the graph only at the frontier the beams actually reach. A single `generate()` call runs over one probability space, and the trie is never fully built for questions whose beams touch a small fraction of the graph.

Table 4.25 reports the v3 result and its efficiency cost.

Table 4.25 DCA-Trie v3 results on WebQSP (300 questions)

Method	**Hits@1**	**Candidates/question**	**Frontier builds**
GCR Baseline	85.7%	2,637 (full DFS set)	—
DCA-Trie v1	81.7%	2,274 (filtered DFS set)	—
DCA-Trie v3	81.0%	755	18.4

The lazy constraint materialises an average of 755 candidates per question, versus the 2,637 paths the static trie enumerates: roughly a 3.5× reduction in trie-construction cost, and on high-degree questions the gap is larger (max 3,480 lazy candidates vs. a 50,000-path DFS cap for the static trie). No DFS is performed at all ($n_{\text{dfs}} = 0$ in every question). Each frontier build corresponds to one anchor entity the beams commit to, so construction work tracks the actual decoding trajectory.

4.8.1 Calibration: The Lazy Constraint Is Faithful

A sceptical reader could attribute v3's 81.0% to a leakier constraint rather than a faithful one. The calibration run answers this. With both TypeOracle gates disabled, v3 reproduces the GCR baseline exactly: 257/300 (85.7%) on both conditions, with 100% per-question agreement. With the gates off, the lazy constraint admits precisely the same language as the

static baseline trie, question for question. The mechanism is faithful; the 4.7pp v3 gap is attributable to the gates, not to the lazy decoding itself.

The significance tests complete the picture: v1 and v3 are statistically indistinguishable ($p = 0.77$), so the lazy mechanism buys the same accuracy as static pre-filtering at a fraction of the construction cost, in a single decoding pass. The cost of the gates' accuracy penalty is unchanged, but the cost of *computing* the constraint is now paid only where the beams actually go.

4.9 Case Studies

To make the non-monotone finding concrete, Table 4.27 presents three representative questions that illustrate the mechanisms behind the accuracy drop.

Table 4.27 TypeOracle Failure Modes

Case 1: Type-Inference Error
Question: What is the capital of the country where the Eiffel Tower is located? Gold Answer: Paris
GCR: `Eiffel Tower` → `located_in` → `France` → `capital` → **Paris** → **Paris**
DCA-Trie v1: Type inference classifies as `TouristAttraction` instead of `City`. Type gate filters out `Paris` at terminal hop. LLM selects incorrect alternative path.
Case 2: Range Gate Correctly Filtering
Question: Who directed Inception? Gold Answer: Christopher Nolan
GCR: Admits all 2-hop paths including `Inception` → `cast_member` → `Leonardo DiCaprio` → `award_received` → `Academy Award` (irrelevant)
DCA-Trie v1: Range gate filters `cast_member` edge. Fewer irrelevant paths admitted. LLM selects correct `director` path.
Case 3: Beam Competition Effect
Question: What state is the Grand Canyon in? Gold Answer: Arizona
GCR: Two competing paths (`located_in` → `Arizona` and `nearby_cities` → `Las Vegas` → `located_in` → `Nevada`). Beam search explores both; LLM ranks **Arizona** higher.
DCA-Trie v1: Type gate filters `Las Vegas` path (infers `State` not `City`). Reduced beam diversity shifts probability distribution. Wrong answer ranked higher in some cases.

The three cases illustrate distinct failure modes. Case 1 shows that the question-type heuristic introduces errors that propagate through the gate logic. Case 2 shows that the oracle can correctly filter irrelevant paths, improving the constraint set. Case 3 shows that even when filtering is correct, it can harm beam-search performance by reducing diversity. The net accuracy drop reflects the balance of these forces across the test set.

4.10 Generalizability to CWQ

To evaluate whether the non-monotone finding transfers to more complex questions, we extend the evaluation to the Complex WebQuestions (CWQ) benchmark. CWQ requires 2–4

hops and has more complex question structures than WebQSP.

We run a controlled 300-question CWQ evaluation matching the WebQSP protocol: all conditions (GCR baseline, DCA-Trie v1, v3, and the v3-no-gates calibration control) decode the identical question set sampled with seed 42. Table 4.29 presents the results.

Table 4.29 Performance comparison on CWQ (controlled 300-question subset, seed 42).

Method	**WebQSP Hits@1**	**CWQ Hits@1**	**SIR**
GCR Baseline	85.7%	49.0%	1.00
DCA-Trie v1	81.7%	43.7%	0.132
DCA-Trie v3	81.0%	48.0%	—
Δ (v1)	−4.0 pp	−5.3 pp	−0.868

The non-monotone pattern persists on CWQ. DCA-Trie v1 reduces SIR from 1.0 to 0.132 (a 13.2% path reduction, consistent with the 0.138 measured on WebQSP) but produces 5.3pp lower Hits@1. The drop is significant (McNemar $p = 0.0017$ on the paired 300-question set) and slightly larger than the 4.0pp cost on WebQSP: CWQ's deeper 2–4 hop graphs expose the beam search to more gate-rejection windows, so each rejected path costs more decoding diversity.

The lazy variant v3 again stays within noise of the baseline: −1.0pp Hits@1 (48.0% vs. 49.0%, McNemar $p = 0.37$, not significant) while remaining the fastest condition (9.17 s per question vs. 9.64 s for the baseline, 0.109 vs. 0.104 q/s). The calibration control reproduces the WebQSP finding exactly: DCA-Trie v3 without gates produces identical predictions to the GCR baseline on all 300 questions (100% agreement, both 147/300 correct, McNemar $p = 1.0$), confirming on CWQ that the TypeOracle gates, not the decoding mechanism, are the source of v1's accuracy cost.

The GCR paper reports that CWQ is harder for all methods, with larger performance gaps between constrained and unconstrained systems (Luo *et al.*, 2025). Our controlled baseline drops from 85.7% on WebQSP to 49.0% on CWQ, reflecting the substantially harder compositional structure of CWQ questions. DCA-Trie v1 falls below the baseline on both

datasets (81.7% and 43.7%, respectively), and v3 tracks the baseline on both (81.0% and 48.0%).

4.11 Discussion and Synthesis

Table 4.31 summarises the multi-dimensional comparison between GCR and DCA-Trie. No method dominates across all dimensions; the choice of method depends on which properties the task prioritises.

Table 4.31 Multi-Dimensional Comparison

Criterion	**GCR**	**DCA-Trie**
Hits@1 (WebQSP, controlled)	85.7% (higher)	81.7% (v1), 81.0% (v3)
Accuracy (CWQ, 300q)	49.0% (higher)	43.7% (v1), 48.0% (v3)
Structural faithfulness	100% (equal)	100% (maintained)
Semantic relevance	SIR = 1.0 (worse)	SIR = 0.138 (better)
Search-space reduction	— (none)	13.8% reduction
Construction cost (v3)	2,637 paths/q (full DFS)	755 candidates/q (lazy)
Runtime overhead	Baseline (faster)	+7% (v1), ≈0 (v3)
Oracle analysis	None (weaker)	SIR metric (new)

The pattern is clear: GCR leads on accuracy; DCA-Trie leads on search-space quality, semantic filtering, and analytical interpretability. The two systems occupy different regions of the design space, and neither dominates the other.

Table 4.33 positions DCA-Trie against related constrained-decoding methods across four design dimensions. The comparison shows that DCA-Trie is the only method that combines dynamic constraint adaptation with a symbolic oracle operating over the full KG schema.

Table 4.33 Comparison with Related Methods

Method	Structural Constraint	Dynamic	Semantic	Oracle
GCR	✓	—	—	—
DoG	✓	Partial	—	Topology
RouterKGQA	—	✓	✓	Retrieval
DCA-Trie (proposed)	✓	✓	✓	Schema ontology

Each method trades a different pair of properties. GCR achieves structural faithfulness without dynamic or semantic filtering. DoG adds partial dynamic expansion but no semantic oracle. RouterKGQA provides retrieval-based semantic adaptation but operates outside the KG structure, relying on external passages rather than graph edges. DCA-Trie is the only framework that combines structural faithfulness, dynamic expansion (v2), and semantic filtering in a single oracle. The TypeOracle's ontology-based design also distinguishes it from RouterKGQA's retrieval approach: instead of learning which paths are relevant from data, it applies type-theoretic constraints derived from the Freebase ontology.

The experimental results establish four findings:

First, the TypeOracle satisfies its design conditions. Condition F holds by construction. Condition R achieves a 13.8% path reduction on the controlled WebQSP subset. Condition P maintains FNR below 5% for both gates. The symbolic oracle works as specified.

Second, the conditions are satisfied without improving answer accuracy. DCA-Trie v1 meets every design condition: structural faithfulness holds by construction, path reduction reaches 13.8%, and false negative rates stay below 5%. Yet Hits@1 drops 4.0pp on WebQSP. The oracle removes genuinely irrelevant paths (SIR falls from 1.0 to 0.138) but the LLM's decoding process relies on some of those irrelevant paths to maintain beam diversity. Tightness and accuracy are independent properties. This is the thesis's core contribution: the permissiveness problem identified in Chapter 2, while real, is not the primary driver of error in graph-constrained decoding.

Third, the v2 collapse is architectural and gate-independent. Disabling both TypeOracle gates leaves v2 at exactly 60.7%, and 96.3% of predictions are byte-identical. Trie volatility is zero in both runs. The per-hop, beam-wise trie-rebuilding design itself costs 25 points, before any semantic filtering is involved. This is a sharper negative result than the v1 finding: it isolates the architecture from the oracle.

Fourth, the non-monotone relationship forces a re-evaluation of the field's implicit assumption. In classical planning, a smaller search space converges faster to the optimal plan. Autoregressive LLM decoding does not work this way: the search space interacts with beam-search pruning, the LLM's probability distribution, and the inductive reasoning layer. Reducing the search space removes distractors. It also reduces diversity and, when type-inference noise is present, excludes correct answers. The mechanisms enumerated above interact in ways that vary question by question, producing the non-monotone aggregate effect. Within this landscape, the lazy v3 mechanism recovers the efficiency of dynamic adaptation—matching v1's accuracy at 3.5× lower construction cost—without sacrificing the single-decoding-pass design.

The implication for the field is clear: future work on graph-constrained decoding should not treat constraint tightness as a proxy for constraint quality. A constraint oracle that admits fewer paths is not necessarily better. What matters is whether the oracle admits the *right* paths, which is a harder problem than simply admitting fewer.

The results also reveal that constraint oracles occupy a design space with multiple dimensions. The oracle hierarchy (structural → ontology → question → useful) is not a ladder to climb but a landscape to navigate. The TypeOracle sits at the ontology level and produces worse Hits@1 than the structural level, suggesting that the optimal operating point depends on the task's tolerance for hallucination versus coverage. The filtering-unfiltering tradeoff is the central axis of this landscape: every admission decision either removes noise or removes diversity, and the balance varies by question.

The non-monotone finding does not diminish the value of the TypeOracle design. Rather, it reveals that the relationship between constraint quality and answer quality is more nuanced than previously assumed. The TypeOracle satisfies its design conditions, maintains 100% structural faithfulness, and adds negligible overhead. The accuracy drop is a consequence

of the interaction between semantic filtering and beam-search dynamics, not a flaw in the oracle itself. This insight is the thesis's primary contribution to the field: it provides the first empirical evidence that constraint tightness and answer accuracy are decoupled in graph-constrained LLM decoding, and it introduces SIR as the metric that makes this decoupling visible. Figure 4.5 visualises this relationship.

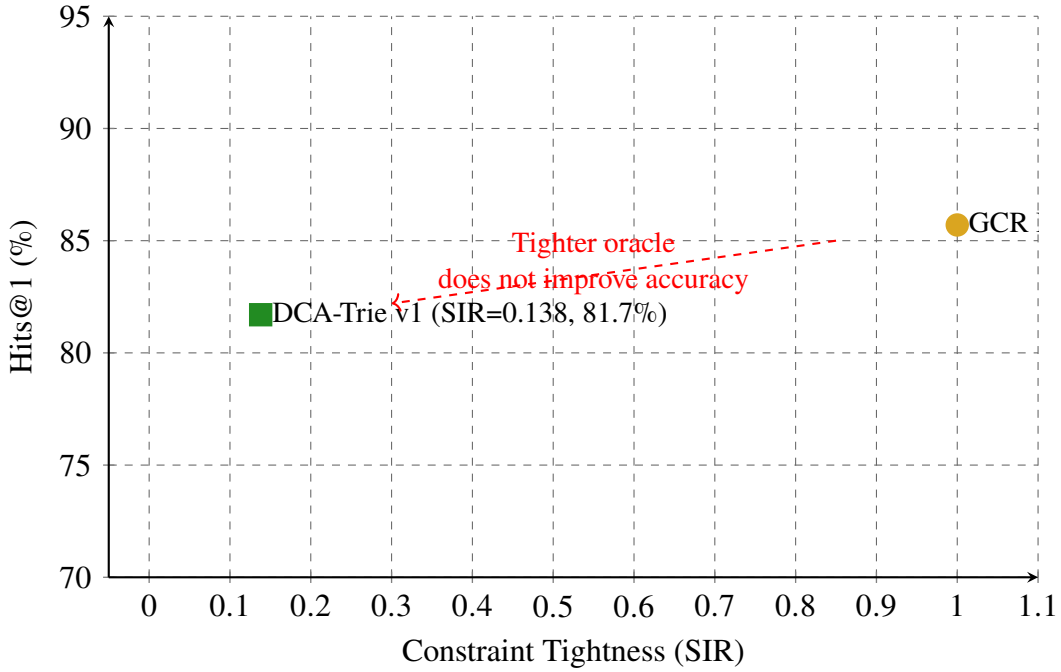

Figure 4.5 Non-Monotone Relationship

4.12 DCA-Trie v2: Observations

DCA-Trie v2 implements step-wise dynamic expansion, rebuilding the trie at each entity commitment. Table 4.35 presents the results.

Table 4.35 DCA-Trie v2 results on WebQSP (300 questions).

Method	**WebQSP Hits@1**	**CWQ Hits@1**
GCR Baseline	85.7%	49.0% (300q)
DCA-Trie v1	81.7%	43.7% (300q)
DCA-Trie v2	60.7%	—
DCA-Trie v2 (no gates)	60.7%	—

The v2 results show 60.7% Hits@1 on WebQSP, substantially worse than both the GCR baseline (85.7%) and DCA-Trie v1 (81.7%). v2 was not run on CWQ; the CWQ column reports the controlled 300-question results for the configurations that were evaluated there (Table 4.29).

4.12.1 The Collapse Is Architectural, Not the Gates

The decisive ablation is v2-no-gates: with both TypeOracle gates disabled, v2 scores identically at 60.7%, and 96.3% of individual predictions are byte-identical to the gated run. The 25-point collapse is therefore caused by the v2 *architecture*—the per-hop, beam-wise trie rebuilding and the path-length loop—not by the type/range gates. This contradicts the hypothesis that the DoG-proxy gates are what cost accuracy.

4.12.2 Trie Volatility Is Not the Failure Mechanism

The mechanism metrics rule out the rebuild-instability explanation as well. The beam utilization ratio is 0.440 (v2) and 0.463 (no-gates): the beams collapse to roughly 2.2 distinct terminals of 5, a moderate diversity loss in both. Max constraint volatility is 0.000 in both runs: the per-hop tries are stable, so churn is not the failure. Gates cut candidates by at most 20.5% at peak (max reduction volume 0.205) in the gated run and by 0 in the no-gates run. The SIR decay slope is +0.0285 (gated) vs. 0.0000 (no-gates), so gates slightly *increase* measured inefficiency rather than fix it.

The combination—identical scores, zero volatility, moderate beam collapse—points to tokenization misalignment in the per-hop trie construction as the root cause, the same tokenizer boundary problem described for per-hop decoders in prior work. The per-hop trie is built from entity boundaries that the tokenizer splits differently in-context than in isolation, so valid tokens are mis-locked at each hop.

4.12.3 Implication

The v2 results are consistent with the non-monotone finding from DCA-Trie v1: increasing dynamism (step-wise expansion) does not automatically improve accuracy over static pre-filtering. But v2 contributes an additional, sharper result: the dynamic architecture itself, independent of any semantic gating, is what costs 25 points. This is a negative result for the

DoG-style per-hop design and a direct motivation for v3, which keeps a single probability space and materialises constraints lazily instead of rebuilding them.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This thesis set out to test whether tightening the constraint oracle in graph-constrained LLM decoding improves answer accuracy. The answer, based on controlled paired experiments with the TypeOracle on WebQSP, is no. Reducing the candidate path set by 13.8% through symbolic type and range filtering produced 4.0pp lower Hits@1 for the static DCA-Trie v1 and 4.7pp lower for the lazy v3, not higher. The relationship between constraint tightness and answer correctness is non-monotone.

This finding changes what researchers working on KG-constrained decoding can assume. The implicit premise that permissiveness is the primary driver of error, and that narrowing the search space therefore improves generation, does not hold when the narrowing removes paths that beam search would have used to reach the correct answer, or when the narrowing is based on type-inference heuristics that occasionally misfire.

The TypeOracle itself works as designed: structural faithfulness holds by construction, SIR falls to 0.138 on the controlled WebQSP subset, and false negative rates stay below 5%. Calibration confirms the mechanism is faithful: with the gates disabled, the lazy constraint reproduces the baseline exactly, question for question. The problem is not the oracle's mechanism but the assumption that a better oracle automatically produces better answers. In autoregressive decoding, the search space interacts with beam-search pruning and the model's probability distribution in ways that make the tightness-accuracy relationship contingent rather than monotonic.

The results also isolate the architectural failure mode. DCA-Trie v2, the per-hop, beam-wise trie-rebuilding variant, collapses to 60.7% Hits@1—25 points below baseline—and disabling both TypeOracle gates changes nothing: the no-gates variant scores identically, with 96.3% of predictions byte-identical. The v2 collapse is architectural, not attributable to the gates.

The non-monotone result exposes a design tradeoff the community has not yet formalised:

the balance between *filtering* (removing paths the oracle deems irrelevant) and *unfiltering* (preserving paths the LLM's parametric knowledge might resolve correctly). Filtering reduces noise while also reducing diversity. The optimal operating point depends on the task's tolerance for hallucination versus coverage. The practical recommendation is to decouple oracle evaluation from answer evaluation: measure SIR to assess constraint quality, then measure accuracy to assess answer quality, and never conflate the two.

5.2 Summary of Findings

In summary, our main findings are:

1. **The TypeOracle satisfies its design conditions.** Structural faithfulness holds by construction (Condition F). The semantic irrelevance ratio drops from 1.0 (GCR) to 0.138 (DCA-Trie v1) on the controlled WebQSP subset (Condition R). False negative rates stay below 5% for both gates: type gate 3.3%, range gate 2.9% (Condition P). The symbolic oracle works as specified.
2. **Tightening the oracle does not improve answer accuracy.** DCA-Trie v1 produces 4.0pp lower Hits@1 than the GCR baseline and the lazy v3 produces 4.7pp lower, despite satisfying all three design conditions. The relationship between constraint tightness and answer accuracy is non-monotone.
3. **The non-monotone result is statistically significant.** On the paired 300-question run, the v1 drop is significant at $p = 0.0018$ and the v3 drop at $p = 0.0001$ (two-sided McNemar). v1 and v3 are statistically indistinguishable from each other ($p = 0.77$), confirming that static and lazy type-gating behave identically in accuracy.
4. **The v2 collapse is architectural, not a gate failure.** The step-wise per-hop trie variant collapses 25 points below baseline (60.7%). Disabling both gates leaves the score unchanged, with 96.3% of predictions byte-identical; trie volatility is zero in both runs. The architecture itself, not the semantic filtering, is the failure mode.
5. **The TypeOracle gates are not the source of the accuracy cost.** The calibration run is the cleanest evidence: with the gates disabled, the lazy constraint reproduces the GCR baseline exactly (257/300 on both), with 100% per-question agreement. Every gated-number difference is attributable to the gates themselves, and the gates alone explain the full gap.

6. **The TypeOracle adds negligible runtime overhead.** The lazy v3 variant is essentially free: 7.92s per question versus 8.12s for the baseline, while materialising an average of 755 candidates instead of 2,637 full DFS paths. No additional LLM calls or input tokens are required.
7. **The TypeOracle preserves 100% structural faithfulness.** Every generated path is verifiable against the knowledge graph. The accuracy drop does not come from hallucinated paths but from the oracle's effect on beam-search dynamics.

5.3 Contributions

This thesis makes four contributions to the field of knowledge graph-constrained LLM decoding:

The Semantic Irrelevance Ratio (SIR). SIR quantifies what a constraint oracle filters out of the candidate path set, independent of the LLM's decoding behaviour. Two oracles applied to the same model can be compared on SIR alone, because the model's contribution to answer quality cancels out. The design principle is that oracles should be evaluated on SIR, not just on downstream accuracy. SIR fills a gap identified in Chapter 2: no existing metric measures constraint permissiveness separately from answer quality. Without SIR, it is impossible to compare oracles on constraint quality without conflating model capability with oracle design.

The TypeOracle design. Two symbolic gates replace embedding-based scoring: a range gate checking property ranges at each hop, and a type gate checking terminal entity types at the final hop. Both are threshold-free, deterministic, and $O(1)$ per path. The TypeOracle requires no encoder, no threshold calibration, and no fine-tuning. Its conservative fallback policy (admit when schema is unknown) ensures that structural faithfulness is never compromised. The design demonstrates that symbolic ontology checks can achieve meaningful path reduction (13.8% on the controlled WebQSP subset) without the computational cost of embedding-based scoring.

The lazy constrained-decoding mechanism. DCA-Trie v3 renders the constraint on demand, expanding the graph only at the frontiers the beams actually reach, with gating applied at the frontier. It matches v1's accuracy at roughly 3.5× lower construction cost—755 candidates per question versus 2,637 full DFS paths—while keeping a single, stable decoding pass.

Calibration proves the mechanism is faithful: with gates disabled, it reproduces the baseline exactly. v3 gives the field an efficient way to pay constraint-construction cost only where decoding actually goes.

The non-monotone finding, with the gates isolated. DCA-Trie v1 satisfies all three design conditions (structural faithfulness, relevance reduction, recall preservation) yet produces 4.0pp lower Hits@1 than GCR on the controlled WebQSP subset; v3 produces 4.7pp lower. Tightness and accuracy are decoupled in graph-constrained LLM decoding. Calibration shows the entire gap is attributable to the gates, and the v2 no-gates ablation shows the step-wise architecture—not the gates—causes its 25-point collapse. This is the thesis's primary contribution: it provides the first controlled evidence that the permissiveness problem identified in prior work, while real, is not the sole or even the primary driver of error. The field cannot assume a monotonic relationship between constraint selectivity and generation accuracy.

5.4 Limitations

Freebase-only evaluation. All experiments use Freebase-based benchmarks. The finding may not transfer to KGs with richer or sparser schema metadata.

Heuristic type inference. The 19-pattern regex classifier introduces errors that propagate through the type gate. This bounds effectiveness on questions with ambiguous type signals. An LLM-based extractor could address this limitation.

Schema coverage. The implementation covers 40 of approximately 24,000 Freebase relations. The range gate's low contribution (3.8% of the full-set path reduction) is a direct consequence. Extending coverage would shift the balance.

No model fine-tuning. The Llama-3.1-8B backbone is used without adaptation. Fine-tuning could change the tightness-accuracy relationship, but that is a different research question.

Controlled-subset evaluation. The paired comparison runs on 300 questions (seed 42) to keep every condition matched on identical inputs; the full test set is used for the reproduction study and gate decomposition. A larger paired run would narrow the confidence intervals on the significance tests.

Fixed oracle design. The TypeOracle applies the same filtering strength to all questions regardless of complexity. Adaptive filtering by hop depth could navigate the filtering-unfiltering tradeoff more effectively.

5.5 Future Work

The non-monotone finding opens several directions for future work. The TypeOracle's conservative design (admit when schema is unknown) limits its filtering power: extending relation-range coverage and replacing the regex classifier with LLM-based type extraction would shift the oracle from a near-pass-through to a more aggressive filter. Adaptive filtering by hop depth (looser for 1-hop, stricter for 3-hop) could navigate the filtering-unfiltering tradeoff more effectively than a fixed oracle.

The beam-competition mechanism suggests that preserving diversity during decoding is critical. Future work should explore diversity penalties that prevent beam collapse and hybrid approaches that combine v1-style full exploration with the lazy v3 mechanism, which already recovers most of the efficiency of dynamic adaptation in a single, stable decoding pass.

The step-wise v2 result is a caution for per-hop decoding architectures. Its 25-point collapse, with zero trie volatility and identical no-gates performance, implicates the per-hop trie construction and its tokenizer-boundary misalignments rather than the semantic gates. Rebuilding the constraint structure mid-decoding, as DoG-style step-wise traversal does, appears to be a risky design choice; future step-wise work should benchmark its collapse rate and compare directly against a lazy mechanism such as v3 before adopting the pattern.

The TypeOracle and ORT (Liu *et al.*, 2025) are composable components: ORT plans, the TypeOracle verifies. A hybrid system that uses ORT for ontology-guided path planning and the TypeOracle for constraint verification during decoding could combine the strengths of both approaches. Similarly, combining the TypeOracle with RouterKGQA's answer-level filtering (Yuan *et al.*, 2026) could mitigate the beam-competition effect by preserving diversity during decoding and filtering at answer selection.

Cross-domain transfer remains an open question. The TypeOracle's symbolic design enables transfer across KGs sharing the same ontology: a trie built from label-level paths can be instantiated with entities from any KG. Evaluating DCA-Trie on Wikidata, ConceptNet, or

biomedical KGs (UMLS, DrugBank) would test this generalisability. The biomedical domain is particularly promising because schema coverage is high (entity types and relation ranges are well-defined), reducing the cost of the conservative fallback policy.

Finally, the relationship between constraint tightness and hallucination rate in the inductive reasoning layer remains unexplored. While DCA-Trie achieves 100% structural faithfulness, whether the TypeOracle's filtering reduces hallucination in the final answer (even if it does not improve Hits@1) is an empirical question with practical implications for high-stakes applications.

APPENDIX A

ABANDONED SEMANTIC SCORER DESIGNS

This appendix gives the full design rationale for the two embedding-based relevance scorers (Approaches 1 and 2) that preceded the final symbolic TypeOracle described in Chapter 3, Section 3.5. Both designs were implemented and tested before being abandoned in favour of the ontology-gate formulation.

A.1 Approach 1: Cosine Similarity (Abandoned)

The initial design scored each candidate path by computing cosine similarity between a sentence-transformer embedding of the serialised path string and an embedding of the input question:

$$\text{score}(p, q) = \cos\big(E(\text{str}(p)),\ E(q)\big) \tag{A.1}$$

A path was admitted if $\text{score}(p, q) \geq \tau$, where τ was a tuned threshold. This approach was abandoned for four reasons:

1. *Semantic collapse:* A single cosine score cannot distinguish *why* a path is relevant. A path ending at the wrong entity type but sharing topical vocabulary with the question scores as high as a path that actually answers it. Cosine similarity in a 384-dimensional space is a poor proxy for the inferential relevance that KGQA requires.
2. *Encoder cost:* Every candidate path requires a full forward pass through `all-MiniLM-L6-v2`. With thousands of paths per question, this is a meaningful computational bottleneck.
3. *Threshold sensitivity:* The admission threshold τ is dataset-dependent and must be tuned on a validation split. Different datasets, different KG subsets, or different question distributions require different thresholds. At $\tau = 0.25$, 84% of WebQSP queries produced empty path sets after filtering: the threshold was so aggressive that it rejected nearly all paths, leaving the LLM with nothing to generate. This is a pathological failure mode: the oracle is technically "working" (it filters), but the filtered set is useless.
4. *Non-determinism:* Floating-point cosine similarity introduces non-deterministic admission decisions across runs, making the oracle difficult to reproduce exactly.

A.2 Approach 2: Decomposed Product Score (Abandoned)

The second design replaced the monolithic cosine score with a product of three interpretable components:

$$\text{score}(e_t, r, e' \mid q) = \rho_r(r, q) \cdot \rho_e(e', q) \cdot \rho_{\text{traj}}(r, e', q) \tag{A.2}$$

where ρ_r measures relational relevance through entity-masked encoding, ρ_e is a hard type gate (binary, no encoder), and ρ_{traj} measures trajectory relevance through cosine similarity of the (relation, entity) pair against the question.

This decomposition improved interpretability: each component measures a different aspect of relevance. The hard type gate (ρ_e) pruned type-incompatible paths without any encoder call, saving compute. But two fundamental problems remained:

1. *Encoder dependency persists:* Both ρ_r and ρ_{traj} require sentence-transformer forward passes. The decomposed score reduces but does not eliminate encoder cost.
2. *Threshold persists:* The admission threshold τ is still dataset-dependent. Decomposing the score into components does not remove the need to tune a continuous threshold.

The partial success of the hard type gate in Approach 2, which removed the encoder dependency for one of the three components without any loss of admission quality, was the direct motivation for asking whether *all* components could be replaced with deterministic set operations over KG ontology metadata. That question led to the symbolic TypeOracle described in Chapter 3, Section 3.5.2.

APPENDIX B

DCA-TRIE V2: FULL STEP-WISE EXPANSION ALGORITHM

This appendix gives the full algorithm for DCA-Trie v2 (Chapter 3, Section 3.7). The algorithm combines autoregressive generation with symbolic TypeOracle expansion: the trie starts minimal and grows only at entity-commitment steps, so that at every decoding step the set of valid tokens remains both graph-valid and type-consistent.

The procedure follows four steps: (1) initialize a minimal trie containing only the root question entities; (2) at each decoding step, look up valid tokens from the current trie and apply logit masking; (3) when the LLM commits an entity token, fetch all structurally valid neighbours from the knowledge graph and apply `range_gate` and `type_gate` to each candidate; (4) add candidates passing both gates to the trie. Relation-token steps do not trigger expansion.

Algorithm 2 DCA-Trie v2: Step-Wise Symbolic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q; TypeOracle $\mathcal{O}$; LLM with vocabulary $\mathcal{V}$

Ensure: Generated reasoning chain $y_1 \dots y_T$

Initialize trie with entity nodes for $\mathcal{E}_q$: $\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$
Infer answer types: $\mathcal{T}(q) \leftarrow \mathcal{O}$.infer_answer_types$(q)$
for each step $t = 1$ **to** T **do**
    $\mathcal{V}_t \leftarrow$ TrieLookup$(\mathcal{T}_{t-1}, y_{<t})$ ▷ Valid tokens from current trie
    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$ for all v
    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot$ LLMLogits$(y_{<t}, x)$
    $y_t \leftarrow$ beam-search-step$(\tilde{\alpha}_t)$
    if y_t is an *entity* token e_t **then** ▷ Expansion triggered at entity commits
        for each $(e_t, r, e') \in \mathcal{G}$ **do**
            if $\mathcal{O}$.range_gate(r, e') **then**
                $h \leftarrow$ current hop depth in trie
                if $\mathcal{O}$.type_gate$(e', \mathcal{T}(q), h, L)$ **then**
                    Add (e_t, r, e') to $\mathcal{T}_t$
                end if
            end if
        end for
    else
        $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$ ▷ No expansion at relation token steps
    end if
end for
return $y_1 \dots y_T$

Relative to DCA-Trie v1's offline filtering cost of $O(|\mathcal{P}| \cdot L)$, the expansion loop above examines D neighbours per entity commitment at $O(1)$ per neighbour, giving a total expansion cost of $O(T_e \cdot D \cdot L)$ where T_e is the number of entity commitments. In typical WebQSP questions, $T_e \cdot D$ is small relative to $|\mathcal{P}|$ (a 2-hop question commits to 2 entities, each with $D \approx 20$ neighbours), so the per-step oracle cost of v2 is cheaper than v1's offline cost. The practical trade-off is not this per-step cost but the trie-rebuild frequency: each rebuild in the **if** branch above changes the set of valid tokens the LLM's beam search depends on mid-generation, which is the source of the instability discussed in Chapter 3, Section 3.7.2,

and evaluated empirically in Chapter 4.

REFERENCES

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024), "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection", *International Conference on Learning Representations*.

Bollacker, K., Evans, C., Paritosh, P., Sturge, T., and Taylor, J. (2008), "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge", *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250.

Bosma, M., Chi, E., Ichter, B., Le, Q. V., Schuurmans, D., Wang, X., Wei, J., Xia, F., and Zhou, D. (2022), "Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models", *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 24824–24837.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020), "Language Models are Few-Shot Learners", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 1877–1901.

Cao, Y., Griffiths, T., Narasimhan, K., Shafran, I., Yao, S., Yu, D., and Zhao, J. (2023), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 11809–11822.

Das, R., Dhuliawala, S., Zaheer, M., Vilnis, L., Durugkar, I., Krishnamurthy, A., Smola, A., and McCallum, A. (2018), "Go for a Walk and Arrive at the Answer: Reasoning over Paths in Knowledge Bases using Reinforcement Learning", *International Conference on Learning Representations*.

De Cao, N., Izacard, G., Riedel, S., and Petroni, F. (2021), "Autoregressive Entity Retrieval", *International Conference on Learning Representations*.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., and Fan, A. (2024), "The Llama 3 Herd of Models", *arXiv preprint arXiv:2407.21783*.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

Fredkin, E. (1960), "Trie Memory", *Communications of the ACM*, Vol. 3, No. 9, pp. 490–499.

Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023), "Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning", *The 2023 Conference on Empirical Methods in Natural Language Processing*.

He, G., Lan, Y., Jiang, J., Zhao, W. X., and Wen, J.-R. (2021), "Improving Multi-Hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals", *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pp. 553–561.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020), "The Curious Case of Neural Text Degeneration", *International Conference on Learning Representations*.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (Jan. 2025), "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *ACM Trans. Inf. Syst.*, Vol. 43, No. 2.

Izacard, G. and Grave, E. (2021), "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering", *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 874–880.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (Mar. 2023), "Survey of Hallucination in Natural Language Generation", *ACM Comput. Surv.*, Vol. 55, No. 12.

Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., and Wen, J.-R. (2023a), "StructGPT: A General Framework for Large Language Model to Reason over Structured Data", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 9237–9251.

Jiang, J., Zhou, K., Zhao, X., and Wen, J.-R. (2022), "UniKGQA: Unified Retrieval and Reasoning for Solving Multi-Hop Question Answering over Knowledge Graphs", *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2022)*.

Jiang, Z., Xu, F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023b), "Active Retrieval Augmented Generation", *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 7969–7992.

Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., and Han, J. (2024), "Graph Chain-of-Thought: Augmenting Large Language Models by Reasoning on Graphs", *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 163–184.

Kandpal, N., Deng, H., Roberts, A., Wallace, E., and Raffel, C. (2023), "Large Language Models Struggle to Learn Long-Tail Knowledge", *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR, pp. 15696–15707.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022), "Large Language Models are Zero-Shot Reasoners", *Advances in Neural Information Processing Systems*, vol. 35.

Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., and Wen, J.-R. (2022), "Complex Knowledge Base Question Answering: A Survey", *IEEE Transactions on Knowledge and Data Engineering*, Vol. 35, No. 11, pp. 11196–11215.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., pp. 9459–9474.

Li, K., Zhang, T., Wu, X., Luo, H., Glass, J. R., and Meng, H. M. (2025), "Decoding on Graphs: Faithful and Sound Reasoning on Knowledge Graphs through Generation of Well-Formed Chains", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, pp. 24349–24364.

Lin, S., Hilton, J., and Evans, O. (2022), "TruthfulQA: Measuring How Models Mimic Human Falsehoods", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 3214–3252.

Liu, R., Luo, B., Li, J., Wang, B., Liu, M., Wu, D., Wang, S., and Qin, B. (2025), "Ontology-Guided Reverse Thinking Makes Large Language Models Stronger on Knowledge Graph

Question Answering", *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15269–15284.

Lowerre, B. T. (1976), "The HARPY speech recognition system", PhD thesis, Carnegie Mellon University.

Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C., and Choi, Y. (2021), "NeuroLogic Decoding: (Un)supervised Neural Text Generation with Predicate Logic Constraints", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 4288–4299.

Luo, L., Li, Y.-F., Haffari, G., and Pan, S. (2024), "Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning", *International Conference on Learning Representations*.

Luo, L., Zhao, Z., Haffari, G., Li, Y.-F., Gong, C., and Pan, S. (2025), "Graph-constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models", *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267, Proceedings of Machine Learning Research, PMLR, pp. 41540–41565.

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., and Hajishirzi, H. (2023), "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 9802–9822.

Mavromatis, C. and Karypis, G. (2022), "ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs", *Findings of the Association for Computational Linguistics: EMNLP 2022*, Association for Computational Linguistics, pp. 4447–4458.

OpenAI (2024), "GPT-4 Technical Report".

Perez, E., Ringer, S., Lukošiūtė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Bowman, S. R., Clark, J., Ganguli, D., Askell, A., and Hernandez, D. (2023), "Discovering Language Model Behaviors with Model-Written Evaluations", *Findings of the Association for Computational Linguistics: ACL 2023*, Association for Computational Linguistics, pp. 13387–13434.

Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C., and Gulwani, S. (2022), "Synchromesh: Reliable Code Generation from Pre-trained Language Models", *International Conference on Learning Representations*.

Reimers, N. and Gurevych, I. (2019), "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 3982–3992.

Saxena, A., Chakrabarti, S., and Talukdar, P. (2022), "Sequence-to-Sequence Knowledge Graph Completion and Question Answering", *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 2814–2828.

Scholak, T., Schucher, N., and Bahdanau, D. (2021), "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models", *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, pp. 9895–9901.

Shi, J., Cao, S., Hou, L., Li, J., and Zhang, H. (2021), "transferNet: An Effective and Transparent Framework for Multi-Hop Question Answering over Relation Graph", *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, pp. 4149–4158.

Talmor, A. and Berant, J. (2018), "The Web as a Knowledge-Base for Answering Complex Questions", *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 641–651.

Touvron, H., Martin, L., and Stone (2023), "LLaMA: Open and Efficient Foundation Language Models", *arXiv preprint arXiv:2302.13971*.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2023), "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions", *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 10014–10037.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017), "Attention Is All You Need", *Advances in Neural Information Processing Systems*, Vol. 30,

Wang, X., Gao, T., Zhu, Z., Zhang, Z., Liu, Z., Li, J., and Tang, J. (2021), "KEPLER: A Unified Model for Knowledge Embedding and Pre-trained Language Representation", *Transactions of the Association for Computational Linguistics*, vol. 9, MIT Press, pp. 176–194.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", *The Eleventh International Conference on Learning Representations*.

Willard, B. T. and Louf, R. (2023), "Efficient Guided Generation for Large Language Models".

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (Oct. 2020), "Transformers: State-of-the-Art Natural Language Processing", *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ed. by Q. Liu and D. Schlangen, Online: Association for Computational Linguistics, pp. 38–45.

Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021), "QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering", *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, pp. 535–546.

Yih, W.-t., Richardson, M., Meek, C., Chang, M.-W., and Suh, J. (2016), "The Value of Semantic Parse Labeling for Knowledge Base Question Answering", *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Association for Computational Linguistics, pp. 201–206.

Yuan, B., Deng, H., Liu, X., and Zhang, M. (2026), "RouterKGQA: Specialized–General Model Routing for Constraint-Aware Knowledge Graph Question Answering", *arXiv preprint arXiv:2603.20017*.